

Universitatea POLITEHNICA din București
Facultatea de Automatică și Calculatoare, Departamentul
de Calculatoare



LUCRARE DE DIPLOMĂ

VisioScience3D

Conducător Științific:
Moraru Anca Andreea

Autor:
Dragomir Andrei-Mihai

București, 2025

University POLITEHNICA of Bucharest

Faculty of Automatic Control and Computers,
Computer Science and Engineering Department



BACHELOR THESIS

VisioScience3D

Scientific Adviser:

Moraru Anca Andreea

Author:

Dragomir Andrei-Mihai

Bucharest, 2025

Realizarea acestui proiect nu ar fi fost posibilă fără ajutorul și sprijinul Doamnei Prof.dr.ing Anca Andreea Moraru. Mulțumesc pentru îndrumare, promptitudine și răbdare. Mulțumesc tuturor celorlalți profesori, laboranți și colegi care mi-au facilitat procesul de învățare și mi-au oferit suportul necesar de-a lungul acestor 4 ani, ajutându-mă în a-mi pune bazele solide în domeniul ingineriei și calculatoarelor.

Abstract

Proiectul VisioScience3D vine ca un răspuns la nevoia de a crea un mediu de învățare interactiv și captivant pentru elevii din învățământul preuniversitar ce lipsește de pe piața curentă. Acesta îmbină tehnologia din ziua de astăzi pentru a crea un sistem de învățare bazat pe vizualizări 3D și simulări interactive, care să faciliteze înțelegerea conceptelor complexe din domeniul științelor exacte.

În momentul curent învățarea geometriei, fizicii, chimiei și altor discipline științifice se face prin metode tradiționale, care nu reușesc să capteze atenția elevilor. Prin acest proiect se dorește venirea în întâmpinarea acestei nevoi și oferirea unui mediu de învățare interactiv și captivant care să faciliteze activitatea didactică și să îmbunătățească rezultatele elevilor.

Proiectul VisioScience3D este o aplicație web care permite utilizatorilor să exploreze domeniul științelor exacte prin intermediul simulărilor interactive și vizualizărilor 3D. Acesta oferă o soluție inovatoare și complexă pentru elevi cât în egală măsură și pentru profesori, având posibilitatea să predea și să evalueze elevii prin intermediul acestuia rapid și eficient.

Cuprins

Mulțumiri	i
Abstract	ii
1 Introducere	1
1.1 Context	1
1.1.1 Definirea problemei	1
1.1.2 Obiective	2
1.1.3 Susținere științifică	2
1.2 Soluția propusă	4
1.3 Rezultate obținute	5
1.4 Structura lucrării	5
2 Analiza cerințelor / Motivația proiectului	7
2.1 Plaja de utilizatori	7
2.1.1 Categori	7
2.1.2 Profilul utilizatorului	7
2.2 Motivația proiectului	8
2.3 Dezvoltarea produsului	9
2.4 Cerințe funcționale	9
2.4.1 Cerințe funcționale pentru utilizatorii elevi	10
2.4.2 Cerințe funcționale pentru utilizatorii profesori	10
2.4.3 Cerințe funcționale pentru sistem	11
2.5 Cerințe nefuncționale	11
2.6 Limitări	13
3 Studiu de piață / Soluții existente	14
3.1 Alte soluții existente	14
3.1.1 Colorado Edu	14
3.1.2 invatamate.com	15
3.1.3 Mozaweb.com	16
3.1.4 iqboard.ro	17
3.2 Raportarea la alte soluții	18
3.3 Motivația alegerii VisioScience3D de către utilizatori	18

3.4	Tehnologii și arhitectură	19
4	Tehnologii utilizate pentru front-end	20
4.1	Soluția de front-end	20
4.1.1	Framework principal	20
4.1.2	Framework 3D	22
4.1.3	Modelare 3D	23
4.2	Soluția UI/UX	24
4.2.1	Brandingul proiectului	24
4.2.2	Experiența utilizatorului	27
4.3	Soluția de creare a scenelor 3D	29
4.3.1	Utilizarea tehnologiilor WebGL și Three.js	29
4.3.2	Integrarea cu frontendul	30
5	Tehnologii utilizate pentru back-end	32
5.1	Descrierea arhitecturii	32
5.1.1	Configurarea clusterului	32
5.1.2	Structurarea și crearea fișierelor YAML	33
5.1.3	Gestionarea clusterului	36
5.1.4	Rutarea și gestionarea traficului	37
5.2	Descrierea serviciilor	38
5.2.1	Descrierea API-urilor	40
5.3	Securitate	40
5.4	CI/CD	41
5.4.1	Port forwarding	41
5.4.2	Deployment	41
5.4.3	Metrici / Monitorizare	42
5.4.4	Avantajele folosirii Prometheus	42
5.4.5	Colectare Prometheus	42
5.4.6	Grafana Dashboard	42
5.5	Soluția de baze de date	45
5.5.1	Structura bazei de date	45
5.5.2	Securitatea datelor	46
5.5.3	Integrarea cu back-endul	47
6	Detalii de implementare	48
6.1	Back-end	48
6.1.1	Dezvoltare back-end	50
6.1.2	Conectivitate	51
6.1.3	Procesarea codului în editorul interactiv	51
6.2	Front-end	53
6.2.1	Configurare front-end	53
6.2.2	Dezvoltare front-end	53
6.2.3	Conectivitate	54

7	Scenarii de utilizare	55
7.1	Înregistrare si autentificare utilizator	55
7.2	Explorarea meniul principal	56
7.3	Accesarea secțiunilor educaționale	57
7.4	Interacțiunea cu scenele 3D educaționale	59
7.5	Gestionarea profilului de profesor	60
7.5.1	Crearea de clase și gestionarea elevilor	60
7.5.2	Crearea de teste și gestionarea lor	61
7.5.3	Vizualizarea rezultatelor	61
7.6	Gestionarea contului de elev	62
7.6.1	Intrarea în clasele profesorului	62
7.6.2	Accesarea testelor și vizualizarea rezultatelor	63
7.6.3	Compilatorul interactiv de cod	63
7.6.4	Tabloul de elemente interactiv	65
8	Evaluarea implementării	66
8.1	Introducere	66
8.2	Evaluarea back-endului	66
8.2.1	Testarea back-endului	66
8.2.2	Monitorizarea back-endului	66
8.2.3	Evaluarea performanței back-endului	67
8.3	Evaluarea front-endului	68
8.3.1	Testarea front-endului	68
8.3.2	Monitorizarea front-endului	68
8.3.3	Evaluarea performanței front-endului	68
8.4	Testarea infrastructurii / Platformei	68
8.5	Probleme întâmpinate	69
8.5.1	Backend & Kubernetes Ingress	69
8.5.2	CI/CD Build & Rollout Workflow accesibilitate	69
8.5.3	Epuizarea memoriei GPU din cauza texturilor masive	69
8.5.4	Crearea fluxului de parsare a codului C++	69
8.6	Analiză comparativă pe metrice	70
8.7	Statistici de dezvoltare	70
9	Concluzii și perspective	72
9.1	Concluzii	72
9.2	Dezvoltare viitoare	72
A	Anexe	74

Capitolul 1

Introducere

1.1 Context

Educația este un pilon și un domeniu de bază al societății, iar tehnologia joacă un rol din ce în ce mai important în acest sector și e nevoie de o adaptare a pieții din zona educațională oferită încă de la vârste relativ mici ale copiilor. Educația ar trebui să evolueze împreună cu tehnologia și copiii de astăzi să învețe să o folosească individual în scopuri educaționale și să lucreze împreună cu ea și nu împotriva. În special pe acest topic, într-o lume în care platformele sociale și mediul de interacțiune video sunt de bază pentru tineri, este esențial să se dezvolte soluții educaționale care să fie atractive și eficiente în procesul clasic de învățare.

1.1.1 Definirea problemei

În acest context, problema pe care o abordăm este crearea unei platforme educaționale interactive care să integreze tehnologii moderne și simulări 3D, pentru a face din procesul de învățare o experiență captivantă și eficientă pentru elevi de gimnaziu și liceu. Această platformă va permite accesul ușor și rapid la informații complexe de matematică, fizică, chimie, astronomie și informatică.

Studentii vor putea explora concepte științifice și interacționa cu simulări 3D, dar și să participe la teste și evaluări pentru a-și verifica cunoștințele. De asemenea, profesorii vor avea la dispoziție un instrument simplu pentru a crea teste și a gestiona clasele pe care le creează, facilitând procesul de predare și evaluare. Platforma va avea la final un sistem interactiv de navigare, recunoaștere a rezultatelor și răsplătire a progresului utilizatorilor prin gamificare tot folosind interacțiuni cu scene 3D, ceea ce va îmbunătăți

experiența utilizatorilor și va stimula învățarea activă și dinamică.

1.1.2 Obiective

Obiectivele principale ale acestui proiect sunt:

- Crearea unei platforme educaționale interactive care să integreze simulări 3D.
- Folosirea unor tehnologii moderne și ușor de menținut pentru viitorului platformei.
- Dezvoltarea unui sistem de gestionare a testelor și evaluărilor ușor de folosit pentru profesori și elevi.
- Implementarea unui sistem de gamificare pentru a stimula învățarea activă și implicarea elevilor în învățare.
- Asigurarea accesibilității și ușurinței în utilizare pentru elevi și profesori.
- Crearea unui mediu de învățare captivant și eficient care să faciliteze înțelegerea conceptelor complexe.
- Integrarea unui sistem de raportare și monitorizare a progresului elevilor.
- Crearea unei interfețe prietenoase și intuitive care să faciliteze experiența profesorilor în gestionarea claselor, testelor, elevilor și a rezultatelor.

Proiectul determină evoluții importante în stilurile de predare și învățare în România, acoperind o nevoie nesatisfăcută încă de vreo platformă modernă, gratuită și cu potențial de creștere și extindere.

1.1.3 Susținere științifică

Multe discipline STEM (matematică, fizică, chimie, etc) implică concepte abstracte foarte dificil de vizualizat, ceea ce poate scădea interesul elevilor. De exemplu, chimia este adesea percepută ca foarte abstractă deoarece elevii nu pot vizualiza conceptele precum structura moleculară sau reacțiile chimice, ele rămânând la nivel de simboluri într-o ecuație pe o foaie care nu spun prea multe despre realitatea și importanța conceptelor în domeniile științifice și ingineresti avansate.

Integrarea vizualizărilor 3D și a tehnologiilor interactive în predarea disciplinelor STEM este susținută de un număr mare de cercetări recente. Majoritatea au ajuns

la concluzia că reprezentările vizuale și simulările contribuie la înțelegerea conceptelor abstracte des apărute în aceste materii și domenii și mai ales sporesc motivația elevilor.

De exemplu, un studiu derulat în școlile din Cehia a arătat că utilizarea modelelor 3D și animațiilor în predarea științelor a dus la o creștere în implicarea elevilor și în performanța la teste, în special în chimie și biologie [1]. De asemenea, o meta-analiză recentă a concluzionat că lecțiile care includ modele 3D interactive au îmbunătățit de peste 1,6 ori varianta standard de învățare teoretică [4]. Deci vorbim de o creștere de 60% raportată a notelor elevilor din grupul care a învățat folosind vizualizări 3D și învățare pe bază de probleme și a totodată a satisfacției raportată de elevi în sine. Așadar înțelegerea și învățarea conceptelor a fost mai ușoară dar și mai plăcută în cazul grupului supus la astfel de tipuri de studiu, ceea ce justifică existența și dezvoltarea de platforme care susțin și înaintază progresul în domeniu.

Simulările 3D aplicate și în zona de laboratoare interactive au condus nu doar la o înțelegere mai bună a subiectelor, ci și la o retenție îmbunătățită a cunoștințelor de-a lungul timpului [5]. Elevii au raportat un nivel mai ridicat de încredere în propriile abilități și o atitudine mai pozitivă față de învățare, ceea ce e cel mai important aspect pentru a crea o generație care știe să se adapteze și să aibă încredere în propriile capacități.

Chiar și mai mult, numeroase alte cercetări evidențiază importanța predării adaptate stilurilor specifice de învățare ale fiecărui elev. Datele arată că un procent semnificativ dintre elevi învață predominant în mod vizual, ceea ce justifică utilizarea elementelor grafice și a animațiilor în predarea la clasă [2], [3]. Un studiu local desfășurat chiar în România confirmă această tendință, indicând o pondere de aproximativ 48% pentru stilul vizual de învățare dintre toate incluse în analiză, ceea ce subliniază necesitatea diversificării suportului educațional și includerea preponderent a elementelor vizuale în predare [2].

În plus, un raport OCDE ¹ a demonstrat că utilizarea controlată a tehnologiei digitale în procesele educaționale poate conduce la o creștere cu până la 15% a scorurilor obținute de elevi la testele de competențe, comparativ cu metodele clasice [7].

Ca și concluzie, studiile prezentate sugerează și susțin că integrarea vizualizărilor 3D și a simulărilor interactive nu doar crește nivelul de atractivitate a învățării, ci și eficiența ei la nivel individual al elevilor. Proiectul *VisioScience3D* se aliniază cu aceste direcții moderne de predare ca o soluție locală zonei României și care răspunde nevoilor curente din educație ale noii generații de elevi, oferind resurse educaționale care comunică pe

¹Organizația pentru Cooperare și Dezvoltare Economică

limba de azi a lor și se axează pe retenție și modelarea procesului învățării pe termen lung.

1.2 Soluția propusă

Ideea platformei este de a crea un mediu de învățare interactiv care să integreze simulări 3D și tehnologii moderne pentru a face procesul de învățare mult mai ușor. Oferă o gamă de materii care pot fi studiate prin această metodă, dar poate funcționa și ca verifcator ad-hoc al cunoștințelor elevilor după predare sau la clasă. Un elev poate intra rapid și facil să verifice o formulă sau altă informație, iar profesorul poate să creeze teste și să gestioneze clasele de elevi în mod rapid și eficient.

Numele VisioScience3D a fost ales pentru a reflecta scopul platformei și este compus din două cuvinte: "Visio" care se referă în mod trivial la vizual, vedere, iar "Science" care se traduce direct în știință. Această combinație sugerează o platformă care îmbină ideea de vizual cu știința, oferind un nume care reflectă esența platformei și scopul său. Titlul conține și termenul 3D, care subliniază focusul vizualizărilor din platformă, cel de a le crea aproape exclusiv în spațiu tridimensional.

Fiecare rol din procesul educațional (elev, profesor) are posibilitatea de accesa funcțiile de învățare și evaluare. De asemenea, platforma va avea un sistem de gamificare care va recompensa elevii pentru progresul lor și va încuraja participarea activă. Opțiuni de vizualizare de tabele de rezultate vor fi disponibile pentru profesori. Elevii vor putea să-și urmărească scorul și progresul în timp real la teste și evaluări și se pot întoarce la conținutul educațional pentru a-și revizui cunoștințele dacă e nevoie.

Testele pot fi create prin selectare și mutare de elemente create ca întrebări, răspunsuri, puncte și altele în interfața dedicată secțiunii de creare a testelor, unde există control granular de la nivel de structură a quizului până la nivel de întrebare, răspuns, selecție de imagini, număr de răspunsuri corecte sau punctaje pe răspunsuri. Profesorii pot vizualiza clasele pe care le dețin, elevii care au participat la teste și rezultatele obținute de aceștia. De asemenea, profesorii pot vizualiza și analiza rezultatele elevilor pentru a înțelege mai bine progresul acestora și pentru a adapta metodele de predare în funcție de nevoile fiecărui elev. Această analiză va permite o abordare personalizată a învățării, care reprezintă de fapt scopul principal din zona de evaluare care se propune în platformă. Profesorii sunt și singurii care au access și la sistemul de invitații al elevilor în clase care funcționează direct către contul elevului.

Elevii pot accesa platforma printr-o interfață axată pe interactivitate și foarte intuitivă,

unde pot explora concepte complexe prin simulări 3D și animații interactive. Pentru ei este destinat meniul 3D principal de selectare a materiei unde pot vedea și interacționa cu toată gama de simulări disponibile. De asemenea, după cum am menționat mai sus, elevii pot participa la teste și evaluări pentru a-și verifica cunoștințele. Aceste teste sunt concepute pentru a fi flexibile din toate punctele de vedere, fiind agnostice de domeniu și oferind o experiență de învățare eficientă, dar fiind și potrivite pentru o testare rapidă după o lecție predată de exemplu.

1.3 Rezultate obținute

Platforma a ajuns într-un punct în care poate fi utilizată de către profesori și elevi pentru a explora concepte și reprezintă o soluție care poate salva timp și poate face învățarea mai rapidă și intuitivă pentru elevii cu stil de învățare vizual, care după cum am menționat și după cum arată studiile sunt majoritari în școlile din România (peste 48% din elevi după cum arată studiile citate). La nivelul de utilizator profesor reprezintă curent o soluție rapidă de testare științifică și monitorizare a elevilor la materii de știință clasice în școala românească, dar și de informatică.

La nivel tehnic, platforma folosește o arhitectură scalabilă a serviciilor din back-end, oferind o scalabilitate foarte ridicată și o disponibilitate crescută datorită separării în microservicii a aplicației și gestionarea traficului separat pe ele.

Arhitectura poate fi ușor extinsă pentru a adăuga noi funcționalități și module, iar platforma poate fi adaptată rapid la nevoile utilizatorilor. De asemenea, partea de front-end este construită folosind tehnologii cu suport extins și comunități mari, ceea ce asigură o suport îndelungat și o posibilă dezvoltare ușoară a platformei în viitor.

1.4 Structura lucrării

Această lucrare este structurată în mai multe capitole, fiecare abordând un aspect diferit al proiectului atât din punct de vedere tehnic, cât și din punct de vedere al implementării și utilizării platformei.

Capitolul 2 oferă o analiză detaliată a cerințelor și a nevoilor utilizatorilor, precum și a profilului utilizatorilor tipici ai platformei. Acesta include o descriere a motivației, a cerințelor funcționale și a celor nefuncționale, precum și a limitărilor și constrângerilor proiectului.

Al treilea capitol (3) analizează piața și competiția din punct de vedere al platformelor

educaționale existente, evidențiind punctele forte și slabe ale acestora și modul în care platforma propusă se diferențiază de cea dezvoltată în cadrul acestui proiect.

Capitolul 4 detaliază tehnologiile utilizate în dezvoltarea platformei, inclusiv limbajele de programare, framework-urile și instrumentele utilizate pentru a crea aplicația. Aici se oferă o privire de ansamblu asupra dezvoltării fiecărei părți importante a aplicației: back-end, front-end (inclusiv UI/UX), scene 3D și bazele de date.

Capitolul 6 oferă detalii despre implementarea platformei, inclusiv bucăți de cod și exemple de dezvoltare a codului, precum și detalii despre configurarea și conectivitatea obținută între diferitele componente ale aplicației.

Cel de-al șaptelea capitol (7) are ca focus definirea și prezentarea scenariilor de utilizare ale platformei, inclusiv modul în care utilizatorii pot interacționa cu aplicația și cum pot crea conturi, teste, clase și cum pot vizualiza scene și experimente 3D. De asemenea, se discută despre modul în care utilizatorii pot accesa și utiliza funcționalitățile platformei, precum și despre modul în care pot beneficia de gamificare și recompense pentru progresul lor.

Capitolul 8 se concentrează pe tipurile de evaluare a platformei, cu accent pe testare și pe modul în care se poate monitoriza sănătatea platformei în cazul lansării către publicul larg. De asemenea se discută și despre evaluare performanței platformei pe diferite niveluri.

Ultimul capitol, 9, oferă concluzii și perspective asupra viitorului platformei, inclusiv posibile îmbunătățiri și extinderi ale funcționalităților existente. De asemenea, se discută despre impactul pe care platforma ar putea să-l aibă asupra educației.

Capitolul 2

Analiza cerințelor / Motivația proiectului

2.1 Plaja de utilizatori

Publicul țintă al aplicației este destul de general, putând fi accesată de orice utilizator care se înregistrează și dorește să învețe sau să își amintească noțiuni de matematică, fizică, chimie, astronomie sau informatică.

2.1.1 Categori

Categoriile principale de utilizatori suportate de platformă sunt utilizatorii elevi și profesorii, fiecare având un rol specific în cadrul aplicației și având acces la funcționalități diferite.

Aceste categorii indică și publicul țintă al aplicației, care este format din elevi de nivel gimnaziu sau liceu și profesori care predau disciplinele menționate la acest nivel de învățământ.

Între aceste două categorii de utilizatori diferă drepturile de acces, tipul de interacțiune cu aplicația dar și funcționalității disponibile.

2.1.2 Profilul utilizatorului

După cum am menționat anterior, aplicația este destinată elevilor și profesorilor de matematică, fizică, chimie, astronomie și informatică. Putem face și o analiză din diverse puncte de vedere demografice și comportamentale:

Vârstă: Aplicația este destinată elevilor de gimnaziu și liceu (10-18 ani) și profesorilor de toate vârstele (25-60 ani).

Tip de învățare: Se folosesc animații 3D pentru a explica conceptele științifice.

Studii: Utilizatorii sunt elevi de gimnaziu și liceu, respectiv profesori cu studii superioare în domeniul educației sau al științelor exacte. Scopul aplicației și publicul țintă explică și nivelul educațional al acestora.

Locație: Aplicația este destinată utilizării în România, iar limba folosită este limba română.

Interese: Utilizatorii au interese în domeniile educației, tehnologiei, științei și învățării interactive, domenii pe care aplicația le acoperă.

Motivație elevi: Elevii doresc să învețe și să își îmbunătățească cunoștințele în domeniile științifice sau să obțină performanțe mai bune la școală și liceu, pentru a obține note mai mari.

Motivație profesori: Profesorii doresc să își îmbunătățească metodele de predare, oferind elevilor o experiență mai interactivă și captivantă.

Tehnologie: Utilizatorii trebuie să aibă cel puțin un nivel de competență tehnologică începător-mediu și să fie familiarizați cu utilizarea platformelor web.

Dispozitive: Aplicația poate fi accesată de pe computere, laptopuri, tablete sau telefoane mobile, având în vedere că este o platformă web.

2.2 Motivația proiectului

Motivația proiectului este de a oferi o platformă educațională interactivă care să ajute elevii, decizia ideii fiind luată după ce s-a studiat piața de soluții existente care oferă acest tip de produs destinat României, în limba română și care oferă o experiență modernă, cât și suport pentru testare încorporat.

Motivația principală este legată direct de necesitatea științifică dezvoltată în capitolul anterior, supimentând nevoia de o asemenea platformă deschisă publicului de elevi și profesori din România, incorporând și o dezvoltare tehnică modernă, cu tehnologiile din ziua de azi, deci cu mult potențial de dezvoltare și menținere ușoară în viitor.

2.3 Dezvoltarea produsului

Dezvoltarea produsului s-a realizat în manieră iterativă, bazat pe testarea progresivă a funcționalităților implementate și feedback-ul primit de la utilizatorii pe care i-am avut la dispoziție în timpul dezvoltării. De fiecare dată s-au reevaluat cerințele și reprioritizat implementarea lor până la realizarea unui MVP ¹ satisfăcător care poate fi menținut și dezvoltat cu alte funcționalități în ultima parte a proiectului.

Cerințele și dorințele clasei țintă de utilizatori au fost colectate prin discuții directe și cu elevii din clasele unde este profesor unul dintre părinții autorului. Elevii au prezentat un entuziasm crescut pentru existența unei astfel de platforme gratuite și au dat sugestii după folosirea prototipului minim funcțional. Printre feedback-ul principal primit la momentul respectiv s-au numărat:

- Animațiile 3D sunt atractive și ușor de utilizat.
- Ar fi utilă o interactivitate mai mare cu scenele.
- Testele sunt eficiente, iar rezultatele sunt clare și ușor de vizualizat.
- O idee interesantă ar fi un sistem de recompense sub formă de obiecte sau insigne 3D pentru realizările utilizatorilor.
- Pe partea de algoritmică, un motor care să vizualizeze cum se modifică structurile de date în timp real ar adăuga un plus de valoare.

Principii de bază ale interfeței:

Interfața a fost realizată având la bază trei principii fundamentale. În primul rând, **simplitatea** a fost esențială, alegându-se un design minimalist, care să nu distragă atenția de la conținutul educațional. Al doilea principiu este **interactivitatea**, iar pentru aceasta au fost integrate animații 3D în cât mai multe zone ale aplicației, pentru a face experiența de învățare mai captivantă și mai plăcută. Alt aspect important a fost **accesibilitatea**, folosindu-se o paletă de culori care să fie ușor de privit și care să nu obosească ochii utilizatorului.

2.4 Cerințe funcționale

Cerințele funcționale ale aplicației:

Cerințele funcționale ale aplicației sunt împărțite în două mari categorii, în funcție de tipul de utilizator. Prima categorie se referă la cerințele funcționale pentru utilizatorii

¹Produs cu funcționalitate minim valabilă

CAPITOLUL 2. ANALIZA CERINȚELOR / MOTIVAȚIA PROIECTULUI 10

elevi, urmate de cerințele pentru utilizatorii profesori. În plus, există și cerințele funcționale pentru sistemul în sine, care definesc modul în care aplicația trebuie să funcționeze și să suporte interacțiunile utilizatorilor.

2.4.1 Cerințe funcționale pentru utilizatorii elevi

Cerințele funcționale pentru utilizatorii elevi sunt cerințe care țin de utilizatorii aplicației și modul în care aceștia pot să interacționeze cu aplicația din poziția de elevi.

Printre cele mai importante funcționalități se numără: **înregistrare și autentificare** (crearea rapidă a unui cont propriu și acces securizat ulterior prin parolă), **navigare în meniul 3D principal** (alegerea materiei dintr-un meniu virtual), **acces la module educaționale** (vizualizarea modulelor structurate prin vizualizări 3D), **interacțiune cu scenele 3D** (interacțiune și observarea în timp real a efectului asupra modelelor), **aderare la clase prin invitație** (acceptarea invitațiilor direct din profil pentru acces la clase), **gestionare invitații** (notificări pentru invitații noi și posibilitatea de a accepta sau refuza direct din panoul profilului), **vizualizare și rezolvare teste** (lista testelor active afișată, cu vizualizarea rezultatelor și acces la cele deschise), **consultare rezultate** (panouri cu scoruri proprii și ale colegilor dacă sunt publice), **administrare profil** (gestionarea conectării sau a datelor personale).

2.4.2 Cerințe funcționale pentru utilizatorii profesori

Cerințele funcționale pentru utilizatorii profesori sunt cerințe care sunt necesare pentru a asigura o experiență adecvată utilizatorilor din rolul de profesor.

Printre funcționalitățile esențiale pentru profesori se numără: **înregistrare cont** (crearea rapidă a unui profil de profesor), **autentificare securizată** (acces pe bază de email și parolă), **meniul 3D principal** (alegerea materiei din interfața virtuală), **acces la resurse educaționale** (consultarea materialelor și a modulelor), **scene 3D interactive** (explorare și manipulare de vizualizări tridimensionale), **gestionare elevi și clase** (creare, editare și organizare grupuri de elevi denumite clase), **gestionare teste** (configurare structură și parametri de evaluare), **deschidere/închidere teste** (controlul perioadelor de acces la evaluări), **vizualizare rezultate** (panouri dedicate cu scorurile elevilor), **analiză și administrare rezultate** (filtrare, export și ajustare note), **meniul invitații** (listarea și urmărirea statusului invitațiilor trimise), **trimitere invitații** (distribuirea directă către elevi din profilul de pe platformă), **administrare profil profesor** (actualizarea datelor și a conexiunii la platformă).

2.4.3 Cerințe funcționale pentru sistem

Cerințele funcționale pentru sistem sunt cerințe care nu țin de utilizatorii aplicației, ci de sistemul în sine și modul în care acesta trebuie să funcționeze. Aceste cerințe sunt esențiale pentru a asigura o experiență de utilizare fluidă și eficientă.

Printre cerințele esențiale pentru sistem se numără: **invitații în timp real** (livrare aproape instantanee către destinatari), **notificări automate** (alertare imediată la primirea unei invitații), **încărcare scene 3D** (randare completă în sub câteva secunde), **latență redusă UI** (răspuns rapid la orice acțiune a utilizatorului), **feedback instant în 3D** (aplicarea imediată a comenzilor de interacțiune), **rezultate pe loc** (afișarea scorului imediat după trimiterea testului), **management utilizatori** (autentificare sigură și administrare eficientă a conturilor), **control starea testelor** (deschidere sau blocare al accesului la evaluări cu efect imediat).

2.5 Cerințe nefuncționale

Dezvoltarea produsului a fost destinată pentru utilizare pe Web, de pe orice dispozitiv cu acces la internet, fără a necesita instalarea de aplicații sau plugin-uri suplimentare. Din natura aplicației, gradul de portabilitate de la dispozitiv la dispozitiv este ridicat.

Interfața trebuie să răspundă rapid la interacțiunile utilizatorilor, iar animațiile 3D să fie fluide și să nu aibă întârzieri semnificative. De asemenea, aplicația trebuie să fie disponibilă pe toate browserele moderne, inclusiv Chrome, Firefox, Safari și Edge.

Pentru o utilizare mai intuitivă sunt adăugate în unele locuri și indicatoari vizuali care arată modul de funcționare. Putem observa și un exemplu în figura următoare, unde utilizatorul este practic îndrumat să pună mouse-ul peste un obiect pentru a activa meniul.

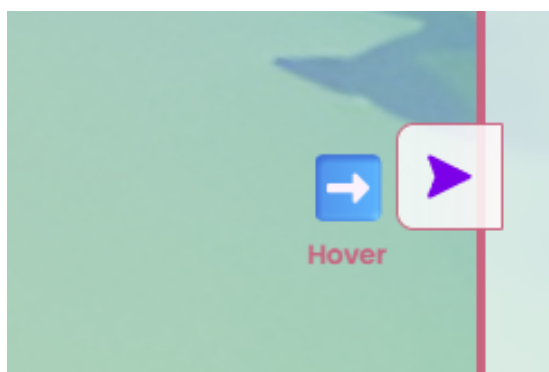


Figura 2.1: Animație intuitivă pentru utilizator deschiderea unui meniu lateral

CAPITOLUL 2. ANALIZA CERINȚELOR / MOTIVAȚIA PROIECTULUI 12

Meniurile conțin informații de prezentare și tutoriale pentru a naviga diferitele simulări 3D, ele fiind primul lucru pe care utilizatorul îl vede la intrarea în diferitele secțiuni. Putem vedea un exemplu în figura următoare.

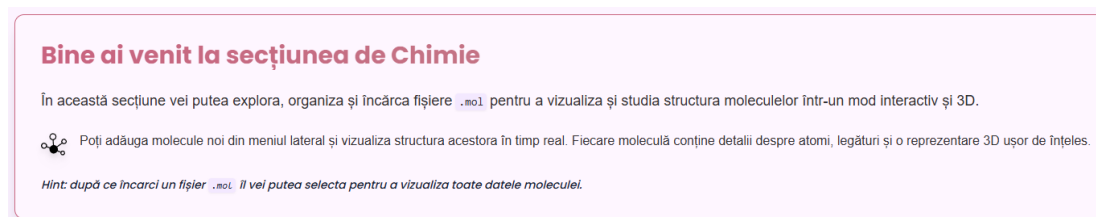


Figura 2.2: Meniu de întâmpinare și ghidare pentru utilizator

Datele afișate în platformă sunt actualizate constant prin comunicare cu serverul care gestionează acel serviciu prin protocolul HTTP.

Ca cerințe nefuncționale necesare pentru încărcarea și gestionarea corectă a scenelor 3D, putem să discutăm pe mai multe planuri, cum ar fi:

1. Browser & API

Aplicația trebuie să ruleze pe browsere moderne care suportă WebGL 1.0 (OpenGL ES 2.0) sau, de preferat, WebGL 2.0 (OpenGL ES 3.0). Browsere compatibile: Chrome > 60, Firefox > 52, Safari > 11, Edge > 16.

2. CPU

- **Minim desktop:** Dual-core 1,5 GHz (Intel Core i3 gen.3+, AMD A6+)
- **Minim mobil/tablet:** Dual-core ARM Cortex-A55 @1,8 GHz

3. Memorie (RAM)

- **Minim:** 2 GB RAM total, din care ≥ 256 MB cel puțin liberi pentru heapul folosit de JavaScript și Three.js pentru încărcat modele și texturi.
- **Recomandat:** 4 GB+ RAM.

4. GPU & VRAM

- **Minim placă grafică integrată:** Intel HD 5000 / AMD Radeon R5 (> 128 MB VRAM)

CAPITOLUL 2. ANALIZA CERINȚELOR / MOTIVAȚIA PROIECTULUI 13

- **Minim placă grafică dedicată:** NVIDIA GT 640 / AMD R7 250 (> 256 MB VRAM)
- **Minim mobil/tablet:** Adreno 306 / Mali-T720 (> 128 MB VRAM)

5. Rețea

- **Lățime de bandă minimă:** 1 Mbps (cu asset-uri comprimate și LOD).
- **Latency:** < 100 ms RTT pentru încărcarea resurselor.

6. Performanță în aplicație

- **FPS țintă:** > 30 fps pe desktop, > 24 fps pe mobil.
- **Timp de încărcare inițială:** < 2 s, < 5 s (la texturile mari).

Aceste cerințe au fost testate în mediul de dezvoltare și sunt valabile pentru majoritatea, putând să difere în mediul de producție în cazul lansării aplicației.

2.6 Limitări

Din punct de vedere al specificului platformei, ea oferă câteva limitări pe mai multe planuri, cum ar fi: **compatibilitate browser și platformă** (aplicația este compatibilă cu majoritatea browserelor de astăzi, însă nu este optimizată pentru browserele mai vechi sau pentru dispozitivele mobile cu specificații mai reduse), **performanță hardware** (aplicația necesită un hardware minim pentru a funcționa corect), **conexiune la internet** (aplicația necesită o conexiune la internet stabilă pentru a funcționa corect), **dispozitive mobile și tabletă** (aplicația necesită un hardware decent pentru rulare, ceea ce este mai greu de obținut pentru dispozitivele mobile), **practice** (aplicația are limitări în utilizare, depinzând de eficacitatea învățării utilizatorului care o folosește), **conținut** (aplicația nu oferă un conținut educațional complet, ci doar partea modelabilă 3D a acestuia pe fiecare materie).

Capitolul 3

Studiu de piață / Soluții existente

3.1 Alte soluții existente

3.1.1 Colorado Edu

Prima soluție analizată este Colorado Edu, o platformă care oferă acoperire pe multiple materii după cum se poate observa în captura următoare.

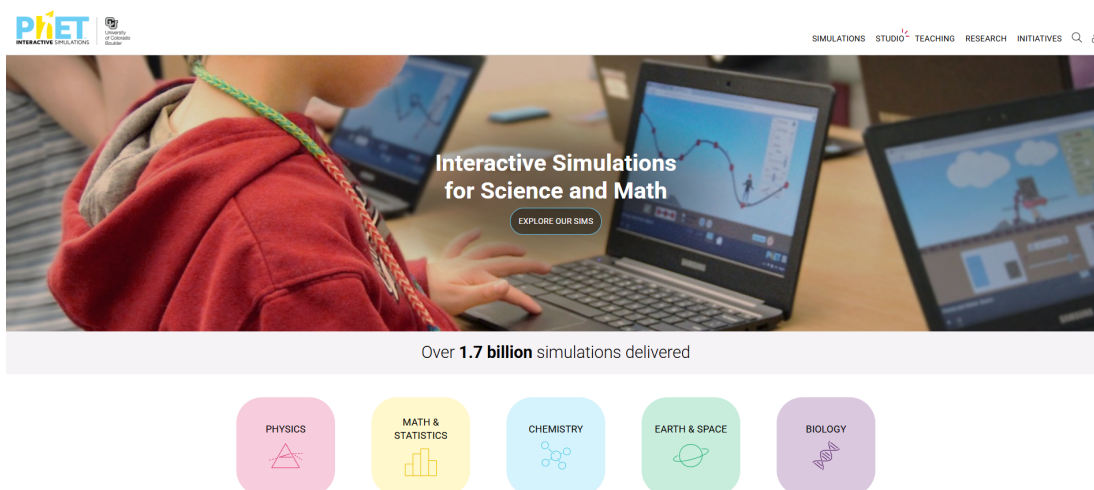


Figura 3.1: Captură de ecran de pe soluția Phet Colorado Edu

Se poate observa designul simplu și intuitiv și suportul relativ extins pe materii. Are și o interactivitate bună la nivel de simulări și alt avantaj e că are acces gratuit. Totuși, nu are un sistem de evaluare integrat și nu permite crearea de teste personalizate. De asemenea, nu are un sistem de management al utilizatorilor, ceea ce face ca utilizarea

platformei să fie limitată la simulări ad-hoc, iar ținta principală ca științelor curriculum este SUA și nu România.

3.1.2 invatamate.com



Figura 3.2: Captură de ecran de pe platforma invatamate.com

Cea de-a doua soluție analizată este invatamate.com, o platformă care oferă o suită de cursuri, vizualizări și simulări online. După cum se poate observa în captura de ecran, platforma are un design relativ învechit și nu foarte atractiv, dar conține multiple resurse utile. Deși numele sugerează că platforma este dedicată matematicii, ea are și simulări de astronomie și joculețe interactive mai generale.

Problema este iar că nu are un sistem de management al utilizatorilor și nici suport pentru evaluare integrat. Lipsa acestor funcționalități face ca platforma să nu fie foarte utilă în mediul educațional, iar utilizarea ei să fie limitată la joculețe simple și triviale. Alte limitări ale platformei sunt că folosește Flash pentru majoritate jocurilor științelor chiar integrări cu Kahoot pentru unele, iar resursele sunt de asemenea legate din surse externe în unele cazuri. De asemenea, experiența nu este cea mai modernă, intuitivă și plăcută după cum se poate vedea în capturile următoare.

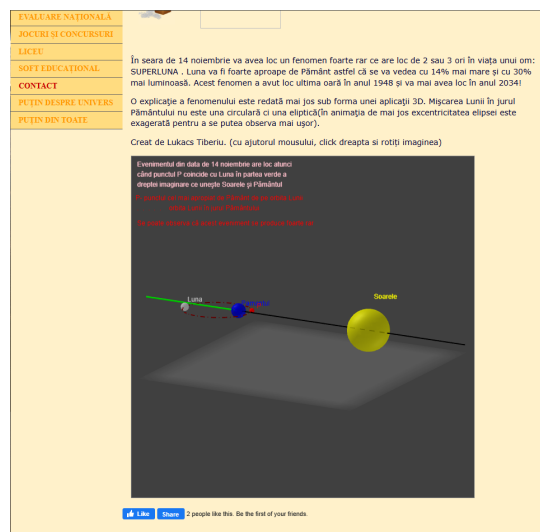


Figura 3.3: Simulări de pe platforma inva-tamate.com

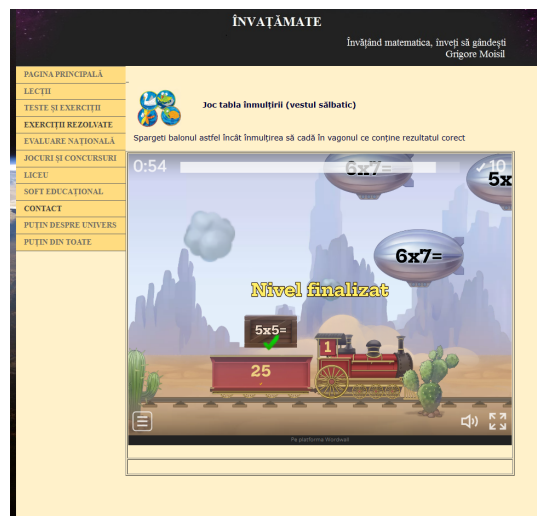


Figura 3.4: Jocuri de pe platforma inva-tamate.com

3.1.3 Mozaweb.com

Această soluție are o arhitectură cu produse pentru mai multe roluri (pentru elevi, pentru profesori, pentru școli) și o experiență de utilizare plăcută și intuitivă. Se poate observa în captura de ecran de mai jos cum arată interfața de întâmpinare a utilizatorului.

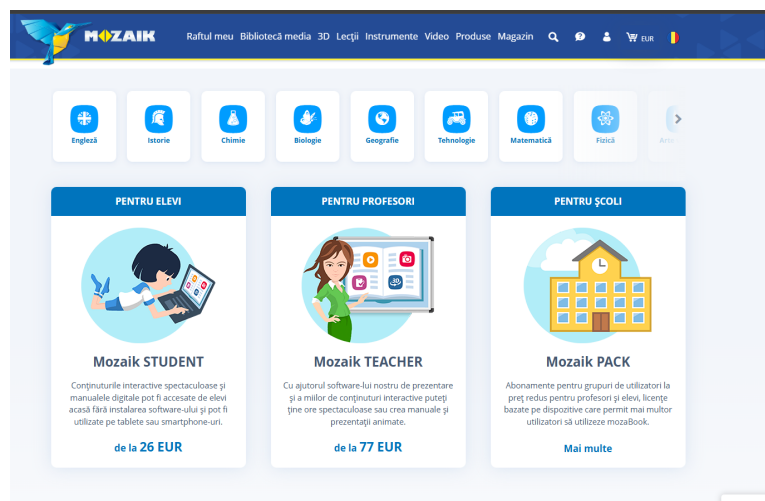


Figura 3.5: Captură de ecran de pe platforma mozaweb.com

Se poate observa suportul pentru numeroase materii și din zona STEM, dar și pentru alte domenii. De asemenea, platforma are simulări foarte profesionale și interactive, dar dezavantajele mari sunt prețurile abonamentelor pentru că discutăm de o platformă comercială și nu gratuită. De asemenea pentru simulări este necesar să se instaleze

un plugin extern, reducând astfel accesibilitatea platformei. Se poate observa mai jos gama de simulări și necesitatea plugin-ului.

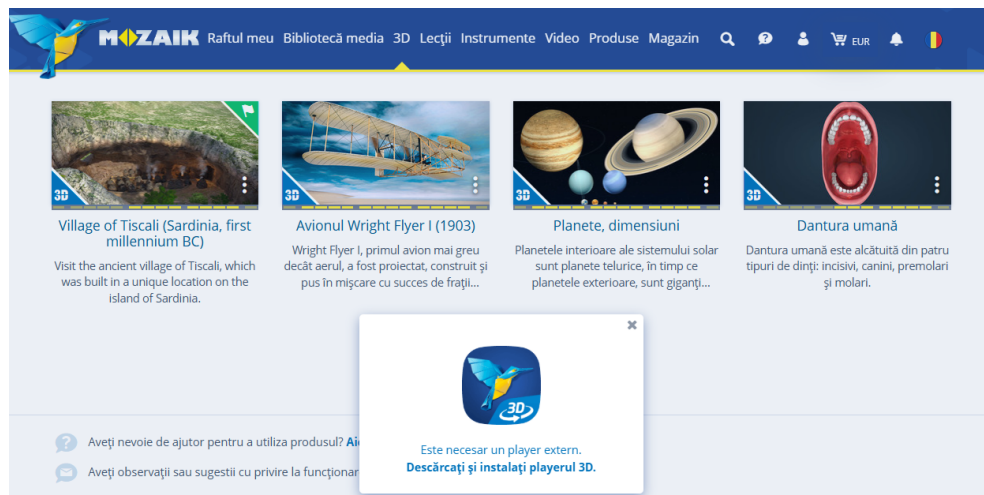


Figura 3.6: Simulări și dovada plugin-ului necesar pentru a le rula

Ținta aceste platforme este una comercială mai mult decât educațională, iar prețurile sunt relativ mari pentru toate tipurile de utilizatori.

3.1.4 iqboard.ro

Ultima soluție analizată este iqboard.ro, o platformă care oferă o suită de resurse educaționale în limba română, cu multiple moduri de lucru, multi touch, bazat pe un sistem de abonamente. Are o interfață interesantă și atractivă și suport extins educațional după cum se poate observa în captura de ecran de mai jos (figura 3.7).

Dezavantajele majore sunt prețurile extrem de mari pentru utilizatori (la nivel de sute de euro după cum se poate observa în figura 3.7). Prețul este relativ nejustificat deși materialele sunt de calitate bună și platforma are un design interactiv și atractiv. În figura 3.8 se poate urmări un exemplu de simulare disponibilă pe platformă. Platforma are moduri separate pentru pregătire, predare și mod desktop, dar nu are un sistem de management al utilizatorilor sau vreun sistem de evaluare sau urmărire a progresului utilizatorilor.

Ca alte funcționalități, platforma se laudă cu modul multi-user care poate fi folosit în modul tabla sau full-screen. După selectarea instrumentului dorit, utilizatorii pot lucra simultan pe tabla, cu modul multi-screens (modul petrecere), cu posibilitate de folosire simultana a doua table interactive (modul MX2) și cu modul de răspuns interactiv.



Figura 3.7: Captură de ecran de pe platforma iqboard.ro

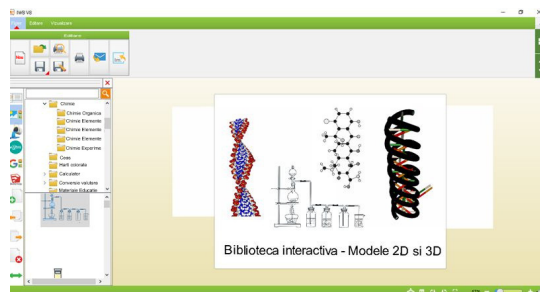


Figura 3.8: Simulări de pe platforma iqboard.ro

3.2 Raportarea la alte soluții

În comparație cu alte platforme, VisioScience3D adresează mai multe nevoi educaționale. În primul rând, ea reprezintă **o singură platformă completă pentru toate materiile STEM** (în timp ce Phet sau Invatamate se concentrează mai mult pe un subiect, VisioScience3D va oferi un mediu unificat cu module interactive de matematică, fizică, chimie, informatică și chiar astronomie, reducând necesitatea schimbării constante de aplicații). De asemenea, alt avantaj e reprezentat de **simulări 3D real-time și configurabile** (Mozaweb de exemplu are mult suport pentru scene 3D, dar ele sunt predefinite și nu pot fi adaptate dinamic; VisioScience3D permite ajustarea parametrilor în timp real, precum forțele, unghiurile, constantele și animarea vectorilor). Un alt avantaj major este **panoul de monitorizare a progresului** (lipsa raportării în timp real este o problemă la majoritatea soluțiilor gratuite; profesorii pot vizualiza instantaneu statistici legate de scoruri, dificultăți întâmpinate și pot adapta testele în funcție de nevoile elevilor). În plus, **gamificarea** (majoritatea platformelor nu au un sistem de gamificare bine definit; VisioScience3D va include elemente de gamificare pentru a spori implicarea elevilor).

3.3 Motivația alegerii VisioScience3D de către utilizatori

VisioScience3D răspunde foarte bine nevoilor profesorilor și elevilor de azi. Oferă simultan simulări 3D real-time pentru toate disciplinele STEM, configurabile direct în browser în platformă, fără pluginuri externe. În timp ce multe platforme existente fie acoperă doar un număr limitat de subiecte, fie implică licențe costisitoare sau

dependențe externe (Mozaweb, IQBoard). VisioScience3D pune la dispoziție un mediu unic, adaptat programei românești. Profesorii pot monitoriza progresul elevilor prin dashboard-uri integrate și primesc rapoarte detaliate. Prin modelul gratis și nivelul simulărilor suportate, VisioScience3D este un ecosistem complet, flexibil și orientat spre rezultatele elevilor.

3.4 Tehnologii și arhitectură

În capitolele ce urmează vom descrie tehnologiile folosite în dezvoltarea platformei, precum și arhitectura acesteia în detaliu. Vom trece prin tehnologiile folosite în zona de front-end, back-end, bazele de date, precum și modul în care acestea interacționează între ele și vom descrie atât conceptual cât și prin diagrame vizuale arhitectura stabilită și implementată pentru fiecare parte menționată.

Capitolul 4

Tehnologii utilizate pentru front-end

4.1 Soluția de front-end

4.1.1 Framework principal

Frameworkul principal care s-a folosit este React JS², care este o tehnologie foarte populară și răspândită bazată pe JavaScript³, una dintre cele mai populare limbaje de programare destinate web-ului existent. În alegerii de tehnologie pentru front-endul platformei VisioScience3D, am concentrat atenția pe criterii precum maturitatea ecosistemului, performanță, curba de învățare, flexibilitate în personalizare și capacitatea de integrare cu biblioteci 3D. Dintre principalele opțiuni (Vue.js, Angular, Svelte), React s-a impus datorită următoarelor avantaje:

Ecosistem matur și susținere largă: React are o comunitate activă de milioane de dezvoltatori, cu un număr impresionant de librării, tutoriale și un suport consistent din partea Meta. Această resursă ne-a permis să adoptăm rapid bune practici și să găsim soluții la provocări specifice dezvoltării 3D. **Model declarativ și component-driven:** React oferă un mod de declarare detaliată în descrierea interfeței, ideal pentru scene 3D complexe, unde starea se propagă predictibil prin componente. Aceasta ușurează managementul ciclului de viață al obiectelor din biblioteca 3D, reducând riscul de actualizări inconsistente.

React Router DOM: Pentru că oferă o soluție de rutare foarte ușoară în navigarea între diferitele secțiuni ale platformei (fizică, chimie, astronomie etc.) prin React Router DOM ce oferă simplitate în definirea rutelor și suportul pentru încărcare întârziată

²<https://reactjs.org/>

³<https://www.javascript.com/>

de module. Alternativele (Vue Router, Angular Router) sunt la fel de capabile, însă integrarea lor cu React Three Fiber, care vom vedea este baza în simulările 3D, ar fi impus un strat suplimentar de interoperabilitate.

Pe zona de stilizare a interfeței, am optat pentru Tailwind CSS¹, un framework CSS utilitar care ne-a permis să creăm un design personalizat. Am comparat framework-uri clasice de CSS (Bootstrap, Material UI) cu Tailwind CSS și am observat următoarele avantaje:

Prioritatea pe utilitare și JIT: Tailwind oferă clase utilitare care permit definirea rapidă a layout-urilor și a stilurilor unice, fără a scrie sutele de linii de CSS personalizat. Motorul JIT (Just-In-Time) generează doar clasele folosite, menținând pachetul final mai mic ca dimensiune.

Consistență și flexibilitate: În loc să se încărce aplicația cu componente predefinite, s-a putut construi o interfață în mod granular respectând paleta de culori și spațiile dorite, dar fără a rescrie componente UI complexe.

Integrare bună cu React: Utilizarea `className` din JSX se potrivește în mod natural cu framework-ul Tailwind. Mai ales plugin-urile pe care le oferă precum `@tailwindcss/forms` facilitează configurarea unui stil constant pentru toate formularele.

Extensibilitate: Tailwind permite personalizarea simplă a temelor, adăugarea ușoară de plugin-uri externe și crearea de componente reutilizabile în UI, fără a sacrifica viteza de dezvoltare.

Bibliotecile de creare și gestionare a graficii 3D sunt esențiale pentru platforma noastră, iar pentru a crea simulări interactive, am ales Three.js², o bibliotecă JavaScript populară. Three.js oferă un API bine gândit pentru crearea de scene 3D, animații și interacțiuni și împreună cu React Three Fiber (R3F)³, o bibliotecă care integrează Three.js cu React, am putut să ne concentrăm pe dezvoltarea rapidă a aplicațiilor 3D fără a ne preocupa de detaliile de implementare ale Three.js. Utilizarea peste a Drei⁴, ce conține o suită de componente și utilitare pentru R3F, a permis accelerarea procesului de dezvoltare, oferind soluții gata făcute pentru anumite probleme comune.

Câteva dintre beneficiile integrării cu Three.js prin React Three Fiber și Drei care au făcut alegerea justificată sunt:

¹<https://tailwindcss.com/>

²<https://threejs.org/>

³<https://r3f.docs.pmnd.rs/r>

⁴<https://drei.pmnd.rs/>

Abstracție declarativă: R3F “împachetează” API-ul imperativ Three.js într-un DSL¹ React, permițându-ne să definim scene, obiecte și materiale în JSX², sub formă de componente. Astfel, putem folosi hook-uri (useFrame, useLoader) pentru a sincroniza animațiile și resursele fără prea mult cod duplicat.

Ecosistem Drei: Biblioteca Drei aduce peste 100 de componente existente (OrbitControls, Text, etc.), accelerate de comunitate, care au economisit timp de implementare.

Optimizări de performanță: Datorită reconciler-ului React, R3F actualizează doar părțile din scenă care se schimbă.

Așadar, combinația React + React Router DOM + Tailwind CSS + React Three Fiber + Drei ne-a oferit un stack unificat, performant și ușor de întreținut, bine adaptat pentru dezvoltarea rapidă de simulări 3D interactive în browser și pentru extinderea ușoară a platformei VisioScience3D. Se poate observa în figura de mai jos cum arată arhitectura simplificată a frontend-ului platformei.

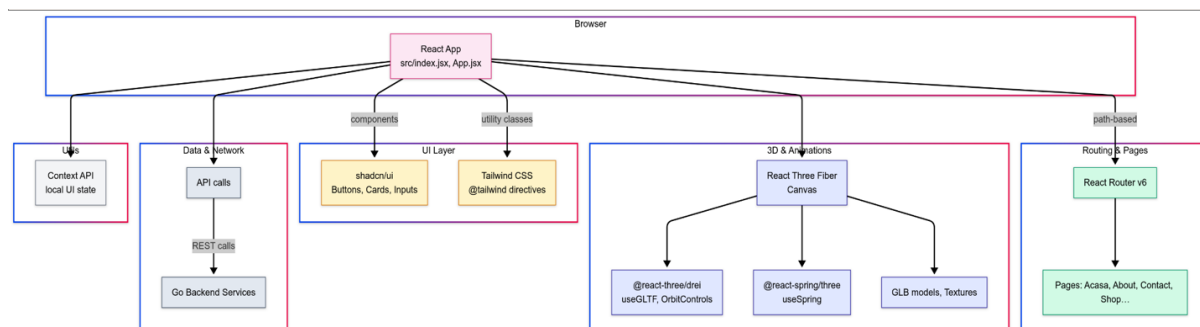


Figura 4.1: Diagrama de design principală a frontend-ului

4.1.2 Framework 3D

După cum am discutat anterior, frameworkul 3D ales pentru acest proiect este React Three Fiber (R3F). Această bibliotecă integrează Three.js cu React, permițându-ne să creăm scene 3D într-un mod declarativ și reactiv. R3F combină principiile WebGL cu capacitățile oferite de React, precum componentizarea și gestionarea stării, ceea ce facilitează dezvoltarea unor scene 3D complexe folosind JSX.

Unul dintre avantajele principale ale R3F este că ne permite să integrăm hook-uri, care îmbunătățesc eficiența și interactivitatea aplicației. De exemplu, hook-ul **useFrame** permite actualizarea scenei la fiecare cadru, ajutând animațiile și interacțiunile în

¹Limbaj specific de domeniu

²Format de JavaScript pentru limbajul de marcare

timp real. **useLoader** este folosit pentru încărcarea resurselor externe, precum texturi sau modele 3D, într-un mod eficient, prin gestionarea încărcării asincrone folosind promisiuni.

Un alt hook important este **useThree**, care oferă acces direct la obiectul Three.js curent, permițându-ne să interacționăm cu scena, camera și renderer-ul. De asemenea, **useRef** este util pentru crearea de referințe la obiectele din scenă, facilitând manipularea acestora în timpul animațiilor sau interacțiunilor.

Pentru gestionarea stării componentelor, folosim **useState**, care permite actualizarea și redarea dinamică a obiectelor din scenă. În plus, **useEffect** ne ajută să gestionăm efectele secundare, cum ar fi adăugarea de evenimente sau actualizarea stării în funcție de modificările din scenă.

Pentru integrarea obiectelor cu extensia .glb am folosit un convertor GLTF¹ care ne permite să convertim modele .glb în componente React Three Fiber oferind grupuri de meshe-uri și materiale. Acest lucru a permis să importăm modelele 3D direct în aplicația React și să creăm diverse manipulări și animații folosind API-ul R3F.



Figura 4.2: Exemplu de convertire a unui model 3D .glb în R3F

4.1.3 Modelare 3D

Am folosit Blender pentru a crea modelele 3D utilizând modele și primitive existente pe internet. De exemplu modelul paginii principale a fost făcut din primitive de insule

¹<https://gltf.pmnd.rs/>

și castele și adaptat la un număr de insule potrivit materiilor din meniu, cât și loc pentru suport viitor pentru alte materii.

Putem observa în figura următoare o scenă din dezvoltarea modelului în Blender.

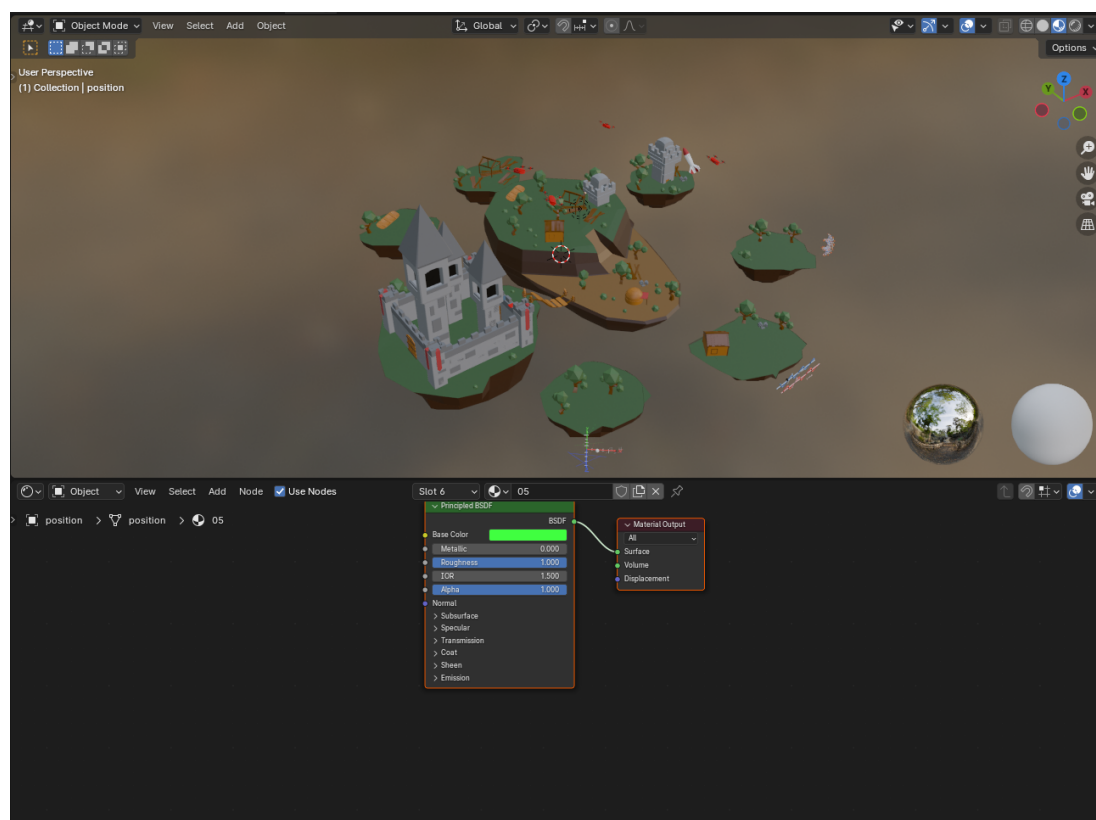


Figura 4.3: Modelul paginii principale în Blender

La acest model s-au adăugat animații de rotație și schimbare a state-ului la mișcarea mouse-ului, animație continuă de plutire și animații realiste de zbor pentru dronele din jurul insulelor.

4.2 Soluția UI/UX

4.2.1 Brandingul proiectului

Paleta de culori

Brandingul VisioScience3D se bazează pe o paletă caldă, prietenoasă și destul de pastelată, menită să creeze o atmosferă de joc și luminoasă pentru elevi. Fundalurile folosesc un gradient simplu de la #fdf4ff (alb-roz pal) prin #f3e8ff (lavandă) spre #fff7ed (piersică pal), în timp ce culorile de accent dau personalitate interfeței: nuanța

de mulberry (#690375) marchează elementele active, bordurile și textele importante, iar tonul maro rozaliu (#AE847E) individualizează secțiunile de chimie și astronomie. Pentru evidențierea vizualizărilor 3D am introdus un violet închis (#4f46e5). Culorile reflectă o atmosferă dinamică și energică, încurajând explorarea și învățarea.

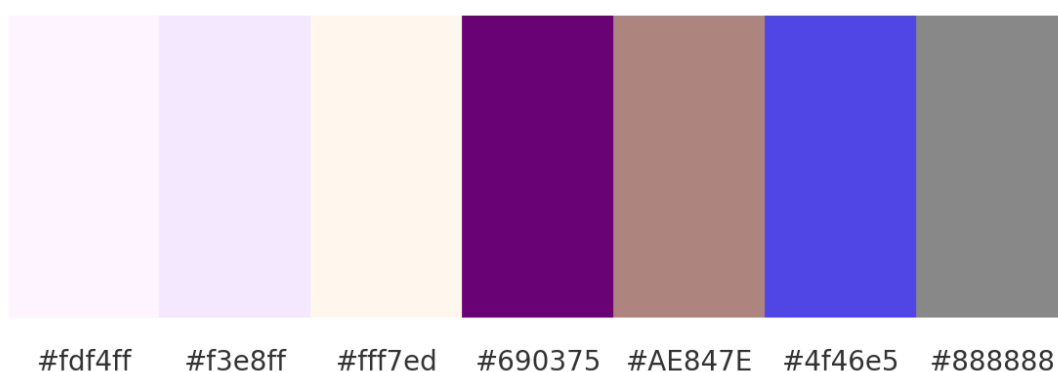


Figura 4.4: Paleta de culori a platformei

Logo-ul

Logo-ul VisioScience3D este relativ simplu și în temă cu paleta de culori, fiind reprezentat de cuvântul "VisioScience3D" scris cu un font reprezentativ și colorat cu un gradient de la mov la mulberry (#690375).

VisioScience3D

Figura 4.5: Logo-ul platformei

Mascota platformei

Mascota proiectului este un balon care se apropie de paleta de culori a platformei și care simbolizează explorarea și învățarea. Balonul apare în toate paginile importante ale platformei și este de fiecare dată animat potrivit acțiunilor paginii respective. De exemplu, pe pagina de înregistrare mascota se mișcă în sus și în jos când utilizatorul scrie în câmpurile de înregistrare, iar pe la înregistrare reușită mascota sare în sus și în jos pentru câteva secunde. Balonul mascotă este și principalul caracter de explorare a meniului principal, învârtindu-se între insule pentru a ajunge la diferitele materii.



Figura 4.6: Mascota platformei

Se poate observa și o captură în timpul animației făcută la succesul la înregistrare.

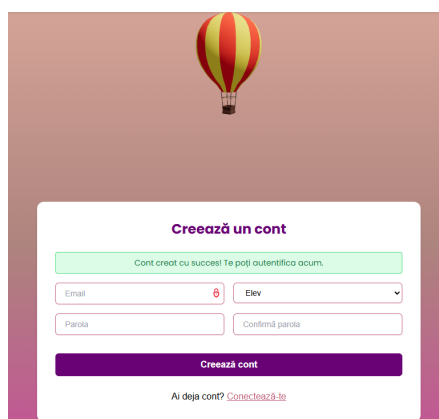


Figura 4.7: Captură de ecran a mascotei la înregistrare

4.2.2 Experiența utilizatorului

Navigarea în platformă

Navigarea în platformă este realizată printr-un meniu principal situat în partea de sus a paginii, iar în meniul 3D prin rotirea balonului mascotă în jurul insulelor pentru a ajunge la diferitele materii. În profil meniul funcționează pe bază de stări și se schimbă în funcție de pagina curentă.

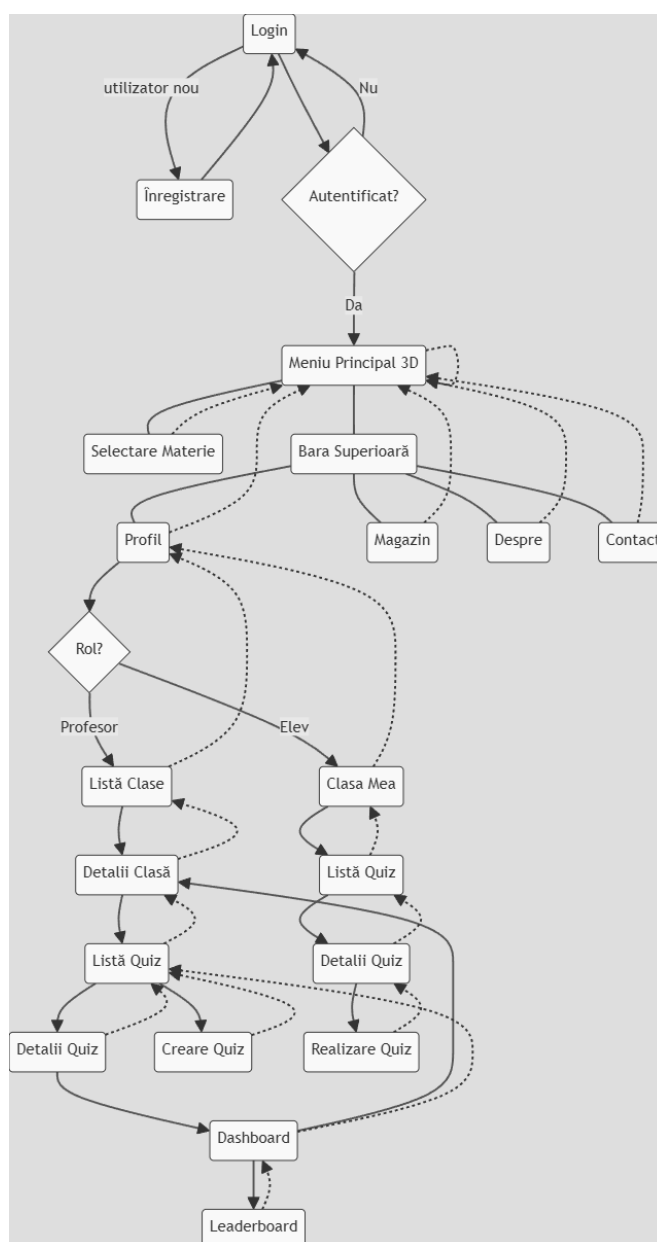


Figura 4.8: Diagrama de navigare a platformei

Gestionarea stărilor

Gestionarea stărilor legate de autentificare și profilul utilizatorului se realizează prin stări locale (`useState`), efecte de ciclu de viață (`useEffect`) și persistență în `localStorage`:

- **Stări locale:** variabilele declarate cu `useState` păstrează răspunsul API-ului, semnul de încărcare și orice mesaj de eroare.
- **Efecte asincrone:** în `useEffect` se trimite cererea HTTP cu token-ul extras o singură dată din `localStorage`, iar la succes se apelează `setUser()`, iar la eșec `setError()`; indiferent de rezultat, în `finally` se setează `setLoading(false)`.
- **Persistență:** token-ul JWT și `userId` sunt citite și stocate o singură dată de la prima montare a componentei, ceea ce permite menținerea autentificării între reîncărcări.

Acest model este simplu dar se poate extinde cu *React Context* sau librării dedicate (*Redux*, *Zustand*) atunci când numărul de componente care împart aceleași stări crește semnificativ.

Designul interfeței utilizatorului

Interfața păstrează un design simplu cu meniul plasat sus și în stânga pentru selecția materiilor după rotire.



Figura 4.9: Interfața meniului principal

Panoul profilului este simplu și ușor de folosit, având elemente care ies ușor în evidență și care sunt ușor de accesat cu acțiuni simple de tipul selecție, drag-and-drop sau butoane de toggle. De exemplu în imagine de mai jos se poate observa interfața profilului elevului.

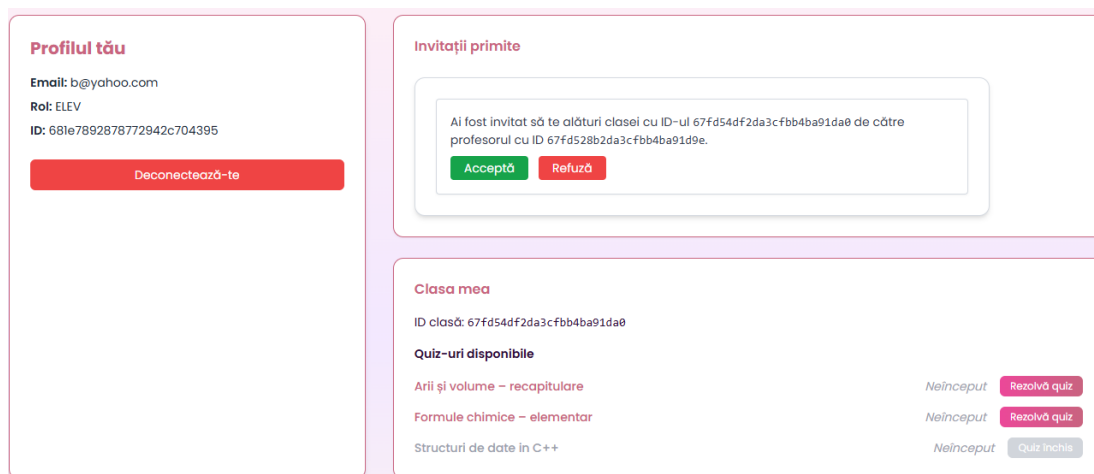


Figura 4.10: Interfața profilului elevului

4.3 Soluția de creare a scenelor 3D

4.3.1 Utilizarea tehnologiilor WebGL și Three.js

Crearea și gestionarea scenelor 3D

Scenele 3D se creează folosind tagul de tip Canvas din React Three Fiber, care ne permite să definim o zonă de desenare 3D în care putem adăuga obiecte, camere și lumini. Acestea sunt gestionate printr-o ierarhie de componente React, fiecare reprezentând un obiect 3D sau un grup de obiecte.

Se folosește tagul `a` din `@react-spring/three` care ne dă acces la grupuri de obiecte și materiale, permițându-ne să transformăm obiecte `glb` în `jsx`.

Interacțiunea cu obiectele 3D

Folosim `useFrame` pentru a actualiza scena la fiecare cadru, simulând callback-urile clasice din loop-urile din jocuri. `UseEffect` e utilizat să actualizeze starea componentelor în funcție de modificările din props, practic folosind `addEventListener` pentru a asculta evenimentele de schimbare a stării pe canvas sau document. `UseRef`, `UseGLTF` și `UseThree` au rolul de a crea referințe la obiectele din scenă, permițându-ne să le manipulăm direct în timpul animațiilor sau interacțiunilor. Folosim callback-uri de bază ca și `stopPropagation` pentru a preveni propagarea evenimentelor mouse-ului de exemplu și `preventDefault` pentru a preveni comportamentele implicite.

Optimizarea performanței graficii 3D

Pentru a optimiza performanța graficii 3D pe framework-ul bazat pe WebGL¹, se abordează câteva tehnici generale:

- **Reducerea numărului de draw calls:** Combinăm obiectele similare folosind `BufferGeometry` pentru a limita costul fiecărui cadru.
- **Frustum culling și occlusion culling:** WebGL elimină automat obiectele aflate în afara câmpului vizual, iar React Three Fiber expune aceste mecanisme nativ, astfel încât obiectele invizibile nu sunt trimise GPU-ului.

React Three Fiber (`@react-three/fiber`) și Drei (`@react-three/drei`) oferă setări și hook-uri dedicate pentru performanță, care sunt esențiale pentru optimizarea scenelor 3D. De exemplu, setarea `<Canvas dpr=[1,1] />` limitează *device pixel ratio* la un interval controlat, prevenind supraîncărcarea GPU-ului pe ecranele cu densitate mare. De asemenea, proprietatea `performance=min:0.5, max:1` ajustează dinamic rata de reîmprospătare a canvas-ului în funcție de activitate, reducând numărul de cadre atunci când scena este statică. Un alt hook important din Drei este `useTexture`, care preîncarcă și cache-uește texturile, evitând alocările repetate și blocările de I/O în bucla de randare.

În proiectul *VisioScience3D*, aceste principii au fost aplicate în mai multe moduri pentru a îmbunătăți performanța aplicației. Texturile au fost decupate și redimensionate la rezoluții de *1k–2k* și convertite în formate comprimate (JPEG/PNG cu bitrate optim), reducând încărcarea pe GPU. Geometria sferică a planetelor utilizează un număr optim de segmente (32–64), echilibrând netezimea cu numărul de vârfuri procesate. Pentru a evita recrearea geometriilor și materialelor la fiecare rerandare, am folosit `React.memo` și `useMemo`. Animațiile sunt gestionate exclusiv pe GPU prin `useFrame`, asigurându-se astfel un randament mai bun. În plus, setările WebGL `toneMapping` și `outputEncoding` au fost ajustate pentru a profita de hardware-ul mai slab și cel mobil, folosind `powerPreference:low-power`. De asemenea, pentru a optimiza performanța graficii 3D, folosim texturi de dimensiuni mici și medii, pentru a asigura stabilitate și un consum redus de resurse.

4.3.2 Integrarea cu frontendul

Pentru integrarea modelelor 3D în platformă, fiecare scenă sau obiect (`.glb`) este încărcat cu declanșatorul `useGLTF` din `react-three/drei` și convertit într-o componentă

¹Framework de bază pentru grafică 3D în browsere, <https://webglfundamentals.org/>

React care reacționează la stările de aplicație. De exemplu, componenta Balloon folosește un `useEffect` pentru a schimba periodic `animationState` într-o listă de stări (`idle`, `swayLeft`, `floatUp` etc.), iar declanșatorul `useSpring` din `react-spring/three` transformă această stare într-un set de proprietăți `position` și `rotation` animate, atribuite mai departe unui element `mesh`.

În plus, componenta de background atașează direct evenimente DOM (`mousedown`, `mousemove`, `keydown`) și gestionate în interiorul declanșatorului `useFrame` din frameworkul de React, permițând controlul direct al rotației scenei în funcție de interacțiunea utilizatorului. Starea și logica aplicației se propagă în lumea 3D, creând o legătură reactivă între UI-ul convențional și grafica gestionată prin WebGL.

Capitolul 5

Tehnologii utilizate pentru back-end

5.1 Descrierea arhitecturii

Backend-ul este construit folosind o arhitectură cu microservicii, care permite scalabilitate și întreținere ușoară a serviciilor care răspund la cereri în spate. Avem patru servicii principale pentru logică de produs, un serviciu care funcționează ca router al arhitecturii, două instanțe de baze de date nerelaționale, un serviciu de gestionare a infrastructurii, un serviciu de monitorizare care funcționează împreună cu un serviciu de provizionare de metrice și un serviciu de vizualizare și utilitarele corespunzătoare pentru gestionarea bazelor de date integrate.

5.1.1 Configurarea clusterului

Clusterul este configurat folosind Kubernetes², care permite orchestrarea containerelor și gestionarea resurselor într-un mod eficient. Fiecare serviciu este containerizat folosind Docker, permițându-ne să rulăm aplicațiile într-un mediu izolat și consistent. Kubernetes gestionează scalarea automată a serviciilor, distribuția sarcinilor și asigură disponibilitatea acestora.

Clusterul a fost creat utilizând kind³, care permite rularea Kubernetes într-un mediu de dezvoltare local. Acest lucru facilitează testarea și dezvoltarea aplicațiilor înainte de a le lansa în mediul de producție.

Dezvoltarea în acest mediu a permis integrarea rapidă a noilor funcționalități și testarea acestora într-un mediu controlat. De asemenea, a permis simularea unor scenarii de

²<https://kubernetes.io/>

³Kubernetes in Docker

scalabilitate și performanță, asigurându-ne că aplicația poate gestiona un număr mare de utilizatori și cereri simultane.

Procesul de implementare și lansare presupune construirea imaginilor Docker¹ pentru fiecare serviciu, încărcarea acestora în registrul de imagini gestionat prin Docker Desktop pentru testare și apoi reîncărcarea acestora în clusterul Kubernetes. Acest proces este automatizat prin intermediul unui pipeline CI/CD în mediul de dezvoltare Github. Pentru asta se folosește GitHub Actions², care permit rularea automată a diferitelor procese de construire, formatare și apoi trecerea prin procesul de testare și implementare/lansare a aplicației. Practic folosind GitHub Actions, am putut să creăm un flux de integrare și livrare continuă³ care a permis să automatizăm procesul de construire, testare și implementare a platformei.

Fiecare serviciu are propriile fișiere `.yaml`⁴ care definesc după caz modul de rulare, variabilele de mediu, resursele necesare, rutarea, porturile expuse și alte setări specifice. Aceste fișiere sunt esențiale pentru a asigura funcționarea corectă a serviciilor în cadrul clusterului Kubernetes și pentru a permite gestionarea eficientă a acestora.

5.1.2 Structurarea și crearea fișierelor YAML

Pentru fiecare componentă a backend-ului—Deployment, Service, Ingress, ConfigMap, PersistentVolumeClaim—s-au creat fișiere YAML separate în directoarele `./k8s/` ale fiecărui serviciu. Denumirea fișierelor corespunde resursei Kubernetes:

`*-deployment.yaml` pentru kind: Deployment

`*-service.yaml` pentru kind: Service

`*-ingress.yaml` pentru kind: Ingress

`*-configmap.yaml` pentru kind: ConfigMap

`*-pvc.yaml` pentru kind: PersistentVolumeClaim

Au fost aplicate toate manifesturile cu:

```
1 kubectl apply -f ./k8s/
```

¹Tehnologie de containerizare a aplicațiilor software <https://www.docker.com/>

²<https://github.com/features/actions>

³<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

⁴fișiere de configurare, în limbaj de marcaj

Un exemplu de *Deployment* pentru serviciul de evaluare:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: evaluation-service
5   labels:
6     app: evaluation-service
7 spec:
8   replicas: 1
9   selector:
10    matchLabels:
11      app: evaluation-service
12   template:
13     metadata:
14       labels:
15         app: evaluation-service
16     spec:
17       containers:
18         - name: evaluation-service
19           image: dragomir1401/evaluation-service:latest
20           ports:
21             - containerPort: 8080
22           env:
23             - name: MONGO_URI
24               value: "mongodb://mongo-evaluation-service
                  ↪ :27017"
```

Acesta definește un *Deployment* pentru serviciul specificat, cu un singur replica pentru rulare și specifică imaginea Docker care va fi utilizată. De asemenea, definește variabilele de mediu necesare pentru conectarea la baza de date. Este unul dintre exemplele simple de manifest Kubernetes folosit.

Tipuri de resurse și rolurile lor

Service (kind:Service): expune pod-urile pe un IP intern al clusterului:

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: evaluation-service
5 spec:
6   type: ClusterIP
7   selector:
8     app: evaluation-service
9   ports:
10    - port: 8080
11      targetPort: 8080
```

Ingress (kind:Ingress): definește rutare HTTP/S prin controller-ul folosit (Kong):

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: evaluation-service-ingress
5   annotations:
6     konghq.com/strip-path: "false"
7 spec:
8   ingressClassName: kong
9   rules:
10    - http:
11      paths:
12        - path: /evaluation
13          backend:
14            service:
15              name: evaluation-service
16              port:
17                number: 8080
```

ConfigMap & PVC: ConfigMap pentru configurări (de ex. pentru serviciul de provizionare `prometheus.yml`) și PVC pentru volumul serviciului de vizualizare a metricilor în Grafana¹:

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: prometheus-config
5 data:
6   prometheus.yml: |
7     global:
8       scrape_interval: 15s
9       evaluation_interval: 15s
10 ---
11 apiVersion: v1
12 kind: PersistentVolumeClaim
13 metadata:
14   name: grafana-pvc
15 spec:
16   accessModes: [ReadWriteOnce]
17   resources:
18     requests:
19       storage: 1Gi
```

Fiecărei resurse îi corespunde un singur fișier YAML – conținând `apiVersion`, tipul `kind`, metadatele și `spec` – care poate fi aplicat cu **kubectl apply -f <fișier>**, sau automatizat printr-un pipeline de integrare continuă / livrare continuă (CI/CD) cum am discutat anterior.

5.1.3 Gestionarea clusterului

Clusterul este creat și gestionat după cum s-a prezentat în primul rând cu Kubernetes, folosind manifesturi YAML. Pe lângă linia de comandă, am integrat *Portainer CE* pentru o interfață web unificată de administrare. Portainer rulează ca un Deployment sub contul de serviciu `portainer-sa`, căruia i s-a atribuit un ClusterRole cu toate privilegiile și un ClusterRoleBinding aferent. Manifestele corespunzătoare (`portainer-sa`, `portainer-role`, `portainer-deployment`, `portainer-rolebinding`, `service` și `portainer-ingress`) sunt folosite pentru configurare și gestionare a accesului

¹Unealtă de vizualizare a metricilor și a datelor de monitorizare

Portainer în cluster. Interfața Portainer, expusă printr-un Service și un Ingress pe ruta `/portainer`, permite vizualizarea nodurilor, pod-urilor, volumelor și gestionarea configurațiilor (scalare, relivrare, mesaje) fără comenzi complexe, oferind în același timp controale RBAC¹ și monitorizare centralizată.

Se poate vedea în imaginea de mai jos cum arată interfața Portainer pe clusterul produsului:

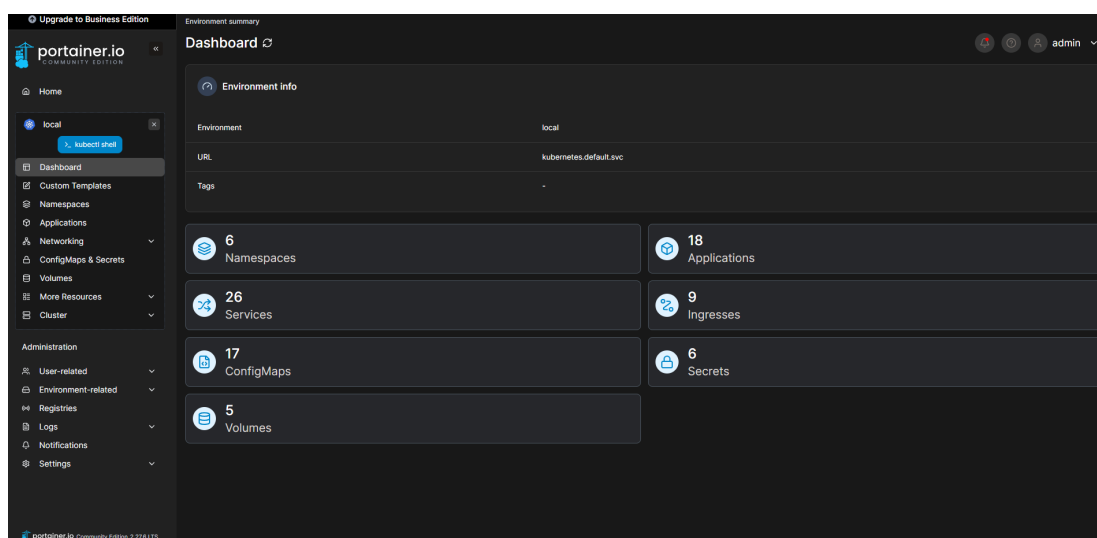


Figura 5.1: Interfata Portainer pe cluster

Si mai jos se poate observa listarea serviciilor in linie de comandă versus în Portainer:

#	Name	Application	Namespace	Protocol	Type	Port(s)	Target	Cluster ID	External IP	Created
1	evaluation-service Details	evaluation-service	default	ClusterIP	8080/TCP	8080	10.86.169.19			2020-04-16 14:20:26
2	redis-0	redis-0	default	ClusterIP	8080/TCP	8080	10.86.170.19			2020-05-20 21:00:07
3	redis-service Details	redis	default	ClusterIP	8080/TCP	8080	10.86.172.19			2020-05-20 17:30:33
4	redis Details	redis	default	ClusterIP	8080/TCP	8080	10.86.170.20			2020-06-18 12:24:14
5	long-controller-metrics	long-controller	long	ClusterIP	102525/TCP 10254/TCP		metrics metrics	10.86.182.46		2020-05-20 14:59:31
6	long-controller-evaluation-webhook Details	long-controller	long	ClusterIP	443/TCP		webhook	10.86.195.74		2020-05-20 14:59:31
7	long-gateway-actix Details	long-gateway	long	ClusterIP	8443/TCP	8443	None			2020-05-20 14:59:31
8	long-gateway-grpc Details	long-gateway	long	NodePort	8083/3040/TCP 8443/3040/TCP	8083 8443	10.86.182.37			2020-05-20 14:59:31
9	long-gateway-nginx Details	long-gateway	long	LoadBalancer	80/8080/TCP	80 8443	10.86.82.8			2020-05-20 14:59:31
10	long-gateway-nginx Details	long-gateway	long	ClusterIP	8083/3040/TCP 8443/3040/TCP	8083 8443	10.86.170.08			2020-05-20 17:30:06

(a) Serviciile în Portainer

PS	DB	FACULTATE	ANUL	Q1	CEINTA	Roadmap	kubectl get pods	READY	STATUS	RESTARTS	AGE
evaluation-service-7f9bf47f47-t8mqc	1/1	Running	0				18d				
feed-data-854c88d4c4-fmrwx	1/1	Running	1 (76dh ago)				18d				
grafana-d6c6c77b6-7knv9	1/1	Running	0				18d				
kong-kong-8787b76dc-vqzjn	2/2	Running	48 (76dh ago)				31d				
mongo-express-feed-data-65b899bf4-1j5n	1/1	Running	15 (76dh ago)				31d				
mongo-express-user-data-6b8899bf4-1j5n	1/1	Running	15 (76dh ago)				31d				
mongo-feed-data-service-65577854dc-17b7x	1/1	Running	16 (76dh ago)				56d				
mongo-user-data-service-59bb7b9895-rvdpvt	1/1	Running	16 (76dh ago)				56d				
monitoring-service-7597c657b6-phm9d	1/1	Running	1 (76dh ago)				18d				
postgres-8594949bfb-8wvnc	1/1	Running	0				76dh				
user-helper-8594949bfb-8wvnc	1/1	Running	0				76dh				
user-intake-8594949bfb-8wvnc	1/1	Running	0				76dh				

(b) Serviciile în linie de comandă

Figura 5.2: Compararea vizualizării serviciilor în Portainer si în linie de comandă

5.1.4 Rutarea si gestionarea traficului

Pentru a centraliza și controla traficul HTTP/S, am ales Kong² ca API Gateway. În cluster am definit o clasă de rutare dedicată, astfel:

¹Controlul accesului bazat pe roluri²<https://konghq.com/>


```
1 kind: IngressClass
2 metadata:
3   name: kong
4 spec:
5   controller: konghq.com/ingress-controller
```

Controller-ul Kong va intercepta toate resursele de tip Ingress care specifică `ingressClassName:kong`. În mediul de dezvoltare, pentru a nu expune un LoadBalancer¹ extern, am folosit `port-forwarding`² local:

kubectl port-forward svc/kong-kong-proxy 8000:80

Astfel, apelurile HTTP către <http://localhost:8000> sunt tunelate prin Kong și pot fi testate rapid în legătură cu serviciile din cluster fără configurări suplimentare de rețea.

5.2 Descrierea serviciilor

În continuare sunt listate serviciile care alcătuiesc backend-ul aplicației, fiecare având roluri și responsabilități specifice în cadrul arhitecturii microserviciilor.

evaluation-service – Serviciul pentru gestionarea logicii de creare, monitorizare, deschide/închidere, ștergere, gestionare de teste și evaluări. Funcționează ca un controller REST API care preia datele din bazele de date, le modelează pentru a fi transmise către frontend și încapsulează și logica de calculare și trimitere către stocarea persistentă de punctaje, scoruri, clasamente și alte statistici.

feed-data – Serviciul care se ocupă de gestionarea datelor legate de secțiunile educaționale și accesarea motorului de generare a moleculelor de chimie. Datele brute din fișierele `.mol`³ ajung în acest serviciu și sunt parsate linie cu linie pentru a identifica poziția atomilor, legăturile chimice dintre ele și tipul lor și mai dimensiunile și tipul moleculelor de asemenea. Datele sunt apoi transmise către routerul serviciului pentru a fi accesibile prin API-ul REST către frontend.

c++-compiler-service – Serviciul care încapsulează logica de gestionare a compilării codului C++ inserat în editorul interactiv și îl împarte în capturi de status a structurilor de date ce apar în cod. Codul este primit din frontend și trece prin mai multe etape de procesare. Se trece linie cu linie prin el pentru a identifica structurile

¹<https://kubernetes.io/docs/concepts/services-networking/>

²<https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

³Fișiere de descriere a moleculelor chimice

de date folosite și a le extrage. Al doilea pas este de creare și adăugare a unui header de urmărire a fiecărei structuri de date capabil să înregistreze și raporteze statusul la fiecare operație. La ultimul pas aceste statusuri sunt adnotate cu metadatele necesare și inserate cronologic într-o listă de evenimente ca să personalizată fi trimisă către frontend.

grafana – Utilitar pentru vizualizarea metricilor și a datelor de monitorizare. Practic este gazdă pentru rularea imaginii de Grafana¹ care primește metricele colectate de Prometheus<https://prometheus.io/> și le vizualizează într-o interfață web.

kong-kong – Controller Kong pentru gestionarea rutării și a API-urilor despre care s-a discutat anterior. **mongo-express-feed-data** – Interfață web pentru gestionarea bazei de date a feed-urilor educaționale. oferă o interfață grafică pentru vizualizarea și manipularea datelor din baza de date MongoDB asociată cu serviciul de feed-uri.

mongo-express-user-data – Interfață web pentru gestionarea bazei de date a utilizatorilor. Oferă ca și cea anterior menționată o interfață grafică pentru vizualizarea și manipularea datelor din baza de date MongoDB asociată cu serviciul de utilizatori.

mongo-feed-data-service – Baza de date MongoDB pentru serviciul de feed-uri educaționale.

mongo-user-data-service – Baza de date MongoDB pentru serviciul de utilizatori.

monitoring-service – Serviciul de monitorizare care colectează și expune metrice. Practic aici se încapsulează partea de cod care folosesc instrumentarea oferită de Prometheus pentru a crea interfețe reutilizabile ce vor fi inserate în zonele din celelalte servicii unde dorim să colectăm metrice.

portainer – Interfață de administrare a clusterului Kubernetes. Oferă toate operațiile posibile pentru gestiune a resurselor din cluster.

prometheus – Serviciul care este gazdă pentru imaginea de Prometheus² folosită în serviciul de monitorizare.

user-data – Serviciul pentru gestionarea datelor utilizatorilor, inclusiv autentificare și autorizare. El funcționează ca un router REST API care are mai multe straturi prin care trec datele. Primul este cel de validare a token-ului de autentificare în middleware-ul JWTAuth, apoi prin parsarea datelor, stocarea lor persistentă în baza de date MongoDB și apoi prin reexpunerea lor către frontend prin API-ul REST.

¹<https://grafana.com/>

²<https://prometheus.io/>

Mai jos putem observa și arhitectura principală de comunicare între serviciile din back-end, `cpp-compiler-service` ne fiind inclus pentru că funcționează pe un flux diferit și nu interacționează direct cu celelalte servicii, ci doar cu frontend-ul.



Figura 5.3: Arhitectura backend-ului aplicației

5.2.1 Descrierea API-urilor

Mai jos sunt principalele grupuri de endpoint-uri fără a le include pe cele de ștergere din rațiuni de dimensiuni ale tabelului. Fiecare serviciu are un API RESTful care permite interacțiunea cu resursele sale. Aceste API-uri sunt definite în routerul fiecărui serviciu care este creat prin biblioteca `Gorilla Mux` în Go. Fiecare serviciu are un set de endpoint-uri care permit operații CRUD (Create, Read, Update, Delete) asupra resurselor sale și interacționează cu baza de date corespunzătoare.

În zona de anexe se pot observa endpoint-urile principale în figura A.1.

5.3 Securitate

Toate API-urile serviciilor sunt protejate prin JWT¹, validat de secțiunea `JWTAuth`, care respinge cererile fără token sau cu token invalid. Traficul extern este dirijat prin Kong după cum am văzut, configurat cu HTTPS/TLS și politici de limitare a numărului de

¹JSON Web Token, folosit în protocoale de autentificare

cereri HTTP, CORS și truncare a căii pentru a preveni atacurile de tip „path traversal”¹ și abuzul de resurse. În interiorul clusterului, rolurile și permisiunile sunt gestionate prin RBAC Kubernetes (ClusterRole/ClusterRoleBinding²), iar Portainer rulează sub un ServiceAccount cu drepturile strict necesare la acces. Conexiunile la bazele MongoDB sunt realizate folosind URL-urile securizate (MONGO_URI) și, în producție, se va face activarea criptării TLS³ și a autentificării la nivel de server de baze de date.

5.4 CI/CD

5.4.1 Port forwarding

În mediul de dezvoltare, pentru a testa rapid Ingress-ul Kong fără LoadBalancer extern, se folosește port-forwarding:

```
1 kubectl port-forward svc/kong-kong-proxy 8000:80
```

Apelurile la <http://localhost:8000> trec prin Kong și pot fi rutate către microservicii.

5.4.2 Deployment

Am automatizat build-ul, testarea și deploy-ul în Kubernetes cu GitHub Actions. Pe scurt:

- **Checkout & Context:** selectăm codul pentru fiecare serviciu și setăm contextul kubectl către kind-visioscience, clusterul unde rulează local mediul de dezvoltare.
- **Build & Push Docker:** compilăm binarele Go, construim imaginea cu build-push-action și o încărcăm pe DockerHub sub dragomir1401/\$SERVICIU:latest.

- **Deploy:** rulăm:

```
1 kubectl apply -f k8s/  
2 kubectl rollout status deployment/${{ steps.extract.  
  ↪ outputs.name }}
```

- Runner-ul GitHub este un hostat pe aceeași mașină ca și clusterul Kind, astfel că are acces direct la kubeconfig⁴.

¹Atacuri bazate pe manipularea căii de acces a resurselor

²<https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

³<https://www.mongodb.com/docs/manual/tutorial/configure-ssl/>

⁴<https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/>

5.4.3 Metrice / Monitorizare

5.4.4 Avantajele folosirii Prometheus

Prometheus este un sistem de monitorizare și alertare foarte popular în mediile moderne de dezvoltare și producție datorită scalabilității și flexibilității sale. Oferă o arhitectură simplă bazată pe colectarea activă a metricilor prin intermediul unor endpoint-uri HTTP expuse de aplicații (format `exposition` format), facilitând integrarea rapidă cu o varietate largă de servicii și microservicii. În plus, arhitectura sa detașată de stare (stateless) și suportul pentru stocarea pe termen scurt cu export către sisteme externe îl fac ideal pentru arhitecturi dinamice, precum Kubernetes. De asemenea, se integrează nativ cu instrumente vizuale precum Grafana, oferind dashboard-uri ușor de configurat și interpretat de către dezvoltatori. În cazul platformei VisioScience3D, Grafana a recunoscut sursa de date din partea Prometheus direct fără configurări suplimentare și s-au putut crea panourile de vizualizare direct prin configurări externe.

5.4.5 Colectare Prometheus

În fiecare serviciu Go am folosit clientul oficial Prometheus:

`github.com/prometheus/client_golang/prometheus` pentru definirea contorilor, histogramei și gauge-urilor.

`github.com/prometheus/client_golang/prometheus/promhttp` pentru expunerea endpoint-ului `/$SERVICIU/metrics`.

De exemplu, în `evaluation-service` în `metrics/metrics.go`:

```
1 registry := prometheus.NewRegistry()
2 HTTPRequestsTotal := prometheus.NewCounterVec(...)
3 registry.MustRegister(HTTPRequestsTotal, HTTPRequestDuration,
   ↪ ActiveEvaluations)
4 http.Handle("/evaluation/metrics", promhttp.HandlerFor(
   ↪ registry, promhttp.HandlerOpts{}))
```

5.4.6 Grafana Dashboard

De exemplu după ce Prometheus trece prin `evaluation_service_http_requests_total`, se crează un dashboard persistent prin volumele Kubernetes. Dashboard-ul Grafana are în general două tipuri de panouri pentru metrice similare. De exemplu pentru metrica de cereri și evaluări active din serviciul de evaluare:

Timeseries(serie temporală de date) pentru `evaluation_service_http_requests_total`.

Gauge(serie de valori) pentru `evaluation_service_active_evaluations`.

Un fragment din JSON-ul dashboard-ului Grafana arată astfel:

```

1  "panels": [
2    {
3      "type": "timeseries",
4      "targets": [{"expr": "
      ↪ evaluation_service_http_requests_total"}],
5      "title": "HTTP_Requests"
6    }, {
7      "type": "gauge",
8      "targets": [{"expr": "
      ↪ evaluation_service_active_evaluations"}],
9      "title": "Active_Evaluations"
10   }
11 ]

```

Aceste panouri sunt importate în Grafana în Evaluation Service Dashboard și actualizate la fiecare 5s (configurabil).

Așadar există trei panouri principale în Grafana pentru trei servicii de logică de produs existente:

Evaluation Service Dashboard pentru `evaluation-service`

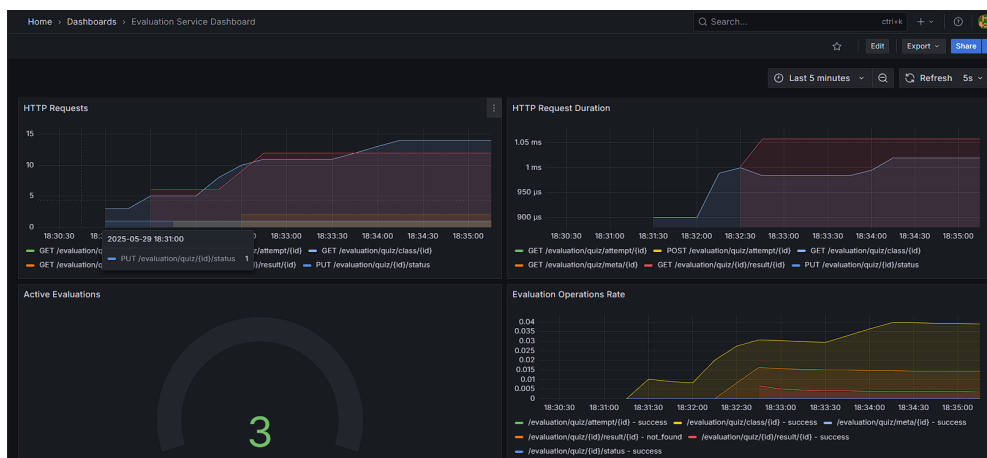


Figura 5.4: Dashboard-ul pentru evaluări în Grafana

Se poate observa logarea numărului de cereri HTTP din serviciul, latența și durata medie a lor, numărul de evaluări active și rata operațiilor pe evaluări.

Feed Data Service Dashboard pentru feed-data

În panou se pot vedea numărul de cereri HTTP, latența și durata medie a lor, numărul de feed-uri active, numărul de molecule active, rata operațiilor asupra feed-urilor și moleculelor, precum și numărul de molecule generate.

User Data Service Dashboard pentru user-data



Figura 5.5: Dashboard-ul pentru metrice de utilizator în Grafana

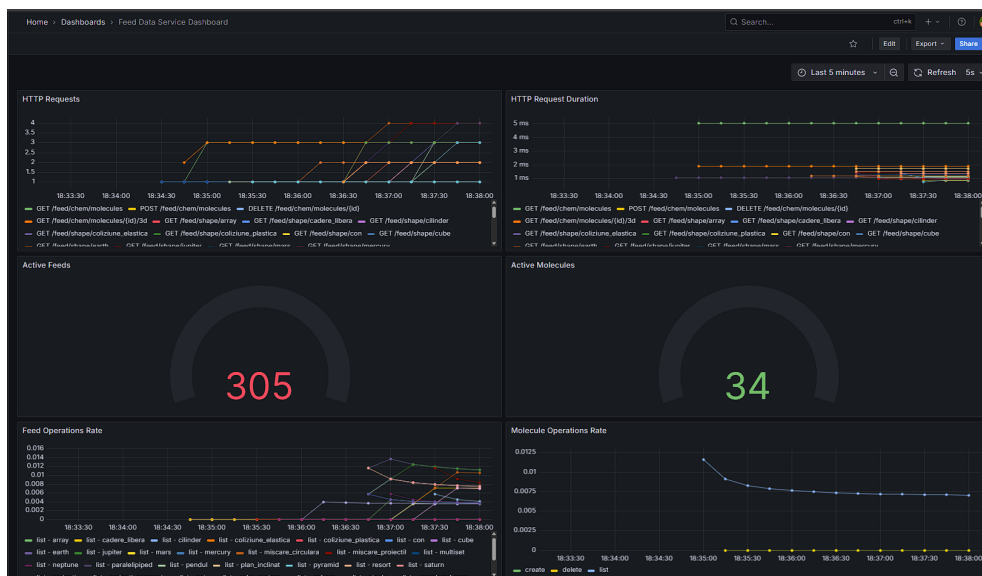


Figura 5.6: Dashboard-ul pentru metrice de feed-uri în Grafana

În panoul pentru metricile din serviciul de utilizatori apar numărul, latența și durata cererilor HTTP, numărul utilizatorilor și al claselor active.

5.5 Soluția de baze de date

5.5.1 Structura bazei de date

Tipuri de baze de date utilizate / Motivația alegerii

Baza de date folosită este MongoDB¹, o bază de date nerelațională care oferă flexibilitate și scalabilitate și e ușor de folosit și mai ales de integrat în microserviciile gestionate în Kubernetes. Am ales o soluție nerelațională deoarece datele noastre sunt semi-structurate și nu necesită relații complexe între tabele, permițându-ne să stocăm documente JSON care pot fi ușor extinse și modificate fără a necesita migrări de baze de date. Structura ar fi putut fi integrată și într-un mediu relațional, dar am considerat că MongoDB oferă o flexibilitate mai mare pentru tipurile de date pe care le gestionăm, cum ar fi feed-urile educaționale, datele utilizatorilor, cât și pentru structura testelor și evaluărilor.

Un alt avantaj al folosirii MongoDB este că permite stocarea datelor într-o formă orientată pe documente similare cu JSON. Alegerea s-a bazat și pe flexibilitatea schemelor, scalabilitatea orizontală și compatibilitatea nativă cu structurile de date folosite, precum și pe posibilitatea de a face agregări complexe și de a gestiona relații parțiale între documente.

MongoDB permite modificarea ușoară a modelelor de date în timp, fără a impune o schemă fixă, aspect important pentru aplicații aflate în dezvoltare continuă și extindere.

Gestionarea datelor

Datele sunt organizate în colecții care corespund entităților sistemului: quiz-uri, formule, molecule, utilizatori, clase și invitații. Fiecare colecție conține documente ce respectă structurile definite în codul backend-ului, folosind driverul oficial MongoDB pentru Go (go.mongodb.org/mongo-driver).

De exemplu, un document din colecția quizzes conține câmpuri precum Title, ClassID și OwnerID (reprezentate prin ObjectID MongoDB), un array de Questions cu întrebări, răspunsuri și punctaje, și un array QuizResults pentru stocarea scorurilor utilizatorilor.

Se folosesc structuri Go care reflectă aceste modele, permițând deserializarea și serializarea automată între BSON² și JSON. Astfel, interacțiunea cu baza de date se face

¹<https://www.mongodb.com/>

²<https://www.mongodb.com/resources/basics/json-and-bson>

prin operații CRUD folosind metodele MongoDB standard, iar aplicația transforma și face validarea datelor ușor.

Gestionarea și vizualizarea datelor în mediul de dezvoltare a fost făcută folosind interfețele oferite de Mongo express, care permit toate operațiile cu documentele din colecțiile MongoDB printr-o interfață web simplă și intuitivă.

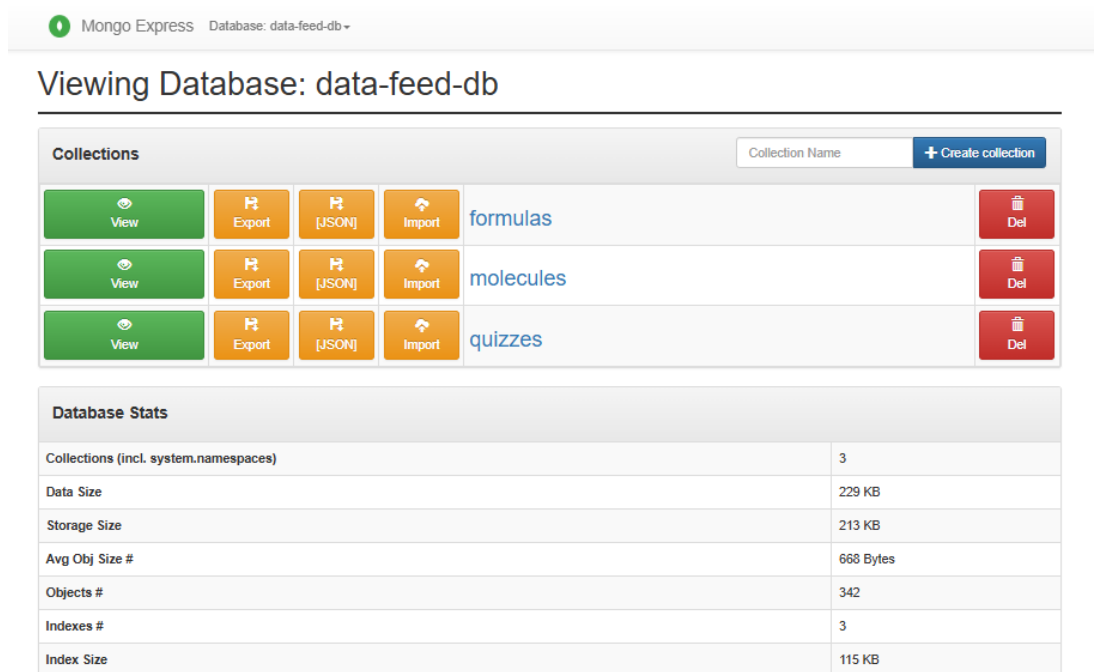


Figura 5.7: Interfața Mongo Express pentru gestionarea colecțiilor

5.5.2 Securitatea datelor

Datele stocate sunt protejate prin controlul accesului la nivel de cluster MongoDB, autentificare și autorizare prin roluri definite (de exemplu, utilizator, profesor). Parolele sunt criptate și gestionate prin variabile de mediu la nivelul serviciilor. Pe viitor se poate face integrarea cu un serviciu de autentificare centralizat (ex: OAuth2, OpenID Connect) sau folosirea unui serviciu de identitate și secrete (precum HashiCorp Vault sau AWS Secrets Manager) pentru a gestiona securitatea și accesul și mai granular la datele sensibile.

Performanța și scalabilitatea bazei de date

MongoDB permite scalarea orizontală prin sharding¹, facilitând distribuția datelor pe mai multe noduri când cantitatea datelor ar ajunge la dimensiuni mari. Indecșii sunt definiți pe câmpuri critice (ex: `_id`, `class_id`) pentru a optimiza căutările și interogările.

De asemenea, conexiunile sunt gestionate eficient prin pool-uri în clientul Go, iar operațiile lungi sunt efectuate cu timeout-uri pentru a evita blocarea serviciilor.

5.5.3 Integrarea cu back-endul

Back-endul este implementat în Go, folosind pachetul oficial MongoDB driver, care oferă suport complet pentru operațiile pe colecții și documente, tranzacții și agregări.

La pornire, fiecare serviciu inițializează o conexiune persistentă la baza de date. Modelele Go definite permit maparea clară între structurile din cod și documentele MongoDB. De exemplu, în `evaluation-service`, obiectele `Quiz` și `QuizResult` sunt manipulate direct în cod, iar metodele HTTP expun API REST care gestionează aceste resurse prin modelul CRUD².

Așadar arhitectura descrisă permite un flux clar și mai ales scalabil între date și servicii, ușurând mentenanța serviciilor și extinderea aplicației.

¹<https://www.mongodb.com/docs/manual/sharding/>

²Create, Read, Update, and Delete

Capitolul 6

Detalii de implementare

În această secțiune se va discuta despre implementarea platformei, acoperind back-endul, front-endul, configurarea, dezvoltarea și conectivitatea acestora.

6.1 Back-end

Pentru back-end s-au analizat diferite tehnologii. Alegerea lor s-a bazat pe comparația descrisă mai jos. Alegerea a fost influențată de mai mulți factori, printre care se numără performanța, ușurința de utilizare și suportul extins pentru dezvoltarea Web . Go oferă un model de concurență eficient și o sintaxă simplă, ceea ce îl face potrivit pentru construirea rapidă de aplicații scalabile. Alt motiv principal a fost compatibilitatea cu infrastructura aleasă pentru proiect, bazată pe Kubernetes și Docker, care se integrează foarte ușor cu Go.

Pe lângă Go, pentru serviciul de compilare a codului C++ s-a optat tot pentru consistență la Go, dar header-ul care este inserat după detectarea structurilor de date pentru a crea raportări de status este scris în C++ pentru a asigura integrare nativă cu codul primit de la utilizator.

```
1 template<typename Container>
2 class ContainerTracer : public Traceable
3 public:
4     ContainerTracer(const std::string& name, Container&
5                     container)
6         : name(name), container(container)
7         { globalTracer->registerObject(this); }
```

```

7
8 class Traceable
9 public:
10     virtual ~Traceable()
11         globalTracer->unregisterObject(this);

```

Listing 6.1: Middleware Prometheus pentru monitorizarea duratei cererilor HTTP

Se poate observa că header-ul se bazează pe structuri generice din C++ pentru a permite orice tip de container să fie integrat în acest tip general de raportare a operațiilor și conținutului.

Mai jos se află comparația limbajelor de care am discutat mai sus.

Tabela 6.1: Compararea limbajelor de programare care ar fi fost potrivite pentru VisioScience3D

Caracteristică	Go ¹	Node.js (JavaScript) ²	Python ³
Performanță	Compilat, foarte rapid, cu suport nativ pentru concurență eficientă (goroutine)	Interpretat, bun pentru I/O non-blocant, dar mai lent decât Go	Interpretat, mai lent, CPU-intensive tasks pot fi mai lente
Concurență	Goroutines ușor de folosit, eficiente din punct de vedere al resurselor	Model event-loop, asincron cu callback-uri/promises/async-await	Suport limitat (threading, asyncio), mai complex pentru concurență
Simplitate și claritate	Sintaxă simplă, tipuri de date clare, cod ușor de întreținut	Flexibil, dar poate duce la cod greu de urmărit (callback hell)	Sintaxă clară, dar structuri mari pot deveni greu de gestionat
Ecosistem	Ecosistem în creștere rapidă, multe librării pentru zona de microservicii, suport bun pentru REST, MongoDB	Ecosistem foarte matur și extins, multe librării, framework-uri	Ecosistem matur, multe librării pentru ML, backend, dar nu neapărat pentru performanță
Scalabilitate	Foarte scalabil, folosit în microservicii și sisteme distribuite	Scalabil pe I/O, dar single-threaded cu limite pe CPU	Scalabilitate limitată, se bazează pe external scaling și caching
Deploy și containerizare	Binar unic compilat, ușor de deployat în containere	Necesită runtime Node.js, imagini mai mari	Necesită runtime Python și librării, imagini relativ mari

Fiecare microserviciu are propria configurație Kubernetes definită prin fișiere YAML pentru Deployment, Service, ConfigMap și alte resurse necesare. Configurarea constă

în containerizarea și automatizarea livrării folosind Docker pentru containerizarea serviciilor și GitHub Actions pentru lanțul CI/CD, care automatizează build-ul, testarea, și deploy-ul pe clusterul Kubernetes local.

Configurarea conexiunilor la baza de date MongoDB este centralizată în helperi, folosind driver-ul oficial MongoDB pentru Go. Monitorizarea serviciilor este asigurată prin integrarea cu Prometheus, cu metrice personalizate definite în cod și expuse la endpoint-uri REST.

```
1 func prometheusMiddleware(next http.Handler) http.Handler
   {
2     return http.HandlerFunc(func(w http.ResponseWriter, r
       *http.Request) {
3         if r.URL.Path == "/feed/metrics" {
4             next.ServeHTTP(w, r)
5             return
6         }
7         start := prometheus.NewTimer(metrics.
           HTTPRequestDuration.WithLabelValues(r.Method,
           utils.NormalizePath(r.URL.Path)))
8         next.ServeHTTP(w, r)
9         start.ObserveDuration()
10        metrics.HTTPRequestsTotal.WithLabelValues(r.
           Method, utils.NormalizePath(r.URL.Path), "200"
           ).Inc()
11    })
12 }
```

Listing 6.2: Middleware Prometheus pentru monitorizarea duratei cererilor HTTP

6.1.1 Dezvoltare back-end

Codul este organizat în pachete separate pentru rute, modele de date, pachet cu fișiere ajutătoare, metrice, utils, panou Grafana, pachet cu fișiere Kubernetes de configurare și middleware (ex: autentificare JWT).

Pentru rutare se utilizează biblioteca Gorilla Mux, iar fiecare endpoint implementează operații CRUD pentru toate resursele din platformă precum quizuri, clase, molecule, formulare și altele. Autentificarea și autorizarea utilizatorilor se realizează prin token-uri JWT, cu middleware dedicat.

Validarea datelor și gestionarea erorilor sunt tratate consistent, cu înregistrare de metrice pentru succes și eșecuri, ceea ce permite monitorizarea calității serviciilor.

Pentru interacțiunea cu MongoDB, structurile Go sunt mapate la documente BSON, cu chei explicite și suport pentru ID-uri generate automat. Structurile BSON sunt convertite nativ în JSON pentru serializare/deserializarea datelor și trimiterea lor către servicii și front-end. Se poate observa un exemplu generare și parsare a token-ului în structuri de date în anexa [A.1](#).

6.1.2 Conectivitate

Microserviciile comunică prin API REST, iar traficul este expus și rulat în interiorul clusterului Kubernetes. Pentru dezvoltare locală, se utilizează port-forwarding Kubernetes pentru a expune temporar serviciile externe către mașina locală. Gestionarea rutelor externe și balansării numărului de cereri se realizează cu ajutorul Kong Ingress Controller, care asigură rutarea traficului HTTP către serviciile potrivite.

În anexa [A.2](#) putem să vedem un exemplu de implementare a unui endpoint pentru crearea unui quiz, care primește datele de la front-end, le validează și le inserează în baza de date MongoDB, iar apoi răspunde cu codul HTTP corespunzător.

Anexa [A.3](#) arată cum se configurează rutele tipice în routerul unui microserviciu, inclusiv aplicarea middleware-ului JWT pentru protejarea endpoint-urilor sensibile.

6.1.3 Procesarea codului în editorul interactiv

Pentru procesarea codului C++ introdus de utilizator în editorul interactiv, s-a implementat un serviciu dedicat care primește codul sursă, îl compilează și execută, returnând rezultatele intermediare în format JSON.

Pentru a vizualiza stările fiecărui container *la runtime* s-au parcurs următorii pași:

1. Inserăm antetul `tracer.h` și instanța globală a tracer-ului în fișierul C++.
2. Funcția `Go TransformCode` parcurge sursa linie cu linie și:
 - detectează declarații de containere STL (`vector`, `map`, `stack`, etc.);
 - generează pentru fiecare un `ContainerTracer` cu `makeTracedContainer()`;
 - inserează apeluri `traceOperation()` după operațiile semnificative (`push_back`, `insert`, `erase`, etc.);

- adaugă `Tracer::getInstance().snapshot()` înainte de **return** și după structuri de control.
3. La rulare, fiecare operație scoate pe `stdout` linii JSON prefixate cu `STATE:`.
 4. Backend-ul Go parsează aceste linii, le convertește în `ExecutionStep` și le trimite front-end-ului.

```

1 #include "tracer.h"
2
3 namespace tracer {
4     Tracer* globalTracer = &Tracer::getInstance();
5 }

```

Listing 6.3: Injectarea antetului și singleton-ului tracer

```

1 std::vector<int> v = {1, 2, 3};
2 auto v_tracer = makeTracedContainer("v", v);    // inserat
          automat de TransformCode

```

Listing 6.4: Generarea unui tracer pentru un vector

```

1 v.push_back(42);
2 v_tracer.traceOperation("insert", "Inserted_element_at_
  back");

```

Listing 6.5: Instrumentarea unei operații asupra vectorului

```

1 for _, line := range strings.Split(rawOutput, "\n") {
2     if strings.HasPrefix(line, "STATE:") {
3         var st models.ExecutionState
4         json.Unmarshal([]byte(line[6:]), &st)
5         output.States = append(output.States, st)
6     }
7 }

```

Listing 6.6: Parsearea ieșirii EXEC și construirea `ExecutionStep`

6.2 Front-end

Pentru dezvoltarea front-end-ului am folosit React, împreună cu React Router DOM¹ pentru navigare, și tehnologia Three.js prin biblioteca React Three Fiber (R3F) și Drei pentru componente 3D reutilizabile. Această combinație a permis crearea unei interfețe interactive cu multiple vizualizări 3D dinamice.

6.2.1 Configurare front-end

Mediul de dezvoltare a fost configurat cu Node.js² și npm³, folosind Vite⁴ ca agregator și server de dezvoltare rapidă. În directorul proiectului, pachetele relevante instalate includ `react`, `react-router-dom`, `@react-three/fiber`, `@react-three/drei` și alte dependențe utile pentru animații și stilizare. Tehnologiile menționate se îmbină cu folosirea Tailwind CSS⁵ pentru stilizare rapidă și eficientă, permițând crearea de interfețe responsive.

6.2.2 Dezvoltare front-end

Componenta principală a aplicației este structurată pe baza React și React Router pentru navigare între pagini. Pentru vizualizările 3D s-au folosit componente R3F, cu suport Drei pentru facilități precum `useGLTF` pentru importul modelelor 3D, `useSpring` pentru animații și `Canvas` pentru spațiul 3D.

Exemple de componente dezvoltate în cadrul proiectului includ **Landing Pages** pentru materii precum Astronomie, Chimie și Informatică, fiecare cu descrieri detaliate și elemente interactive. De asemenea, s-au dezvoltat **formulare**, precum `ChemistryAddForm`, care permit adăugarea moleculelor cu încărcare de fișiere și validări. În ceea ce privește vizualizările 3D, s-au creat componente precum `ChemistryBalloon`, `Drone`, `Island` și `SpaceBackground`, care sunt animate și controlate de utilizator. Alte componente includ elemente **educaționale** pentru structuri de date și concepte de programare, completate cu vizualizări grafice. În plus, s-au dezvoltat panouri de control și pagini pentru quiz-uri, clase, invitații și alte funcționalități, toate acestea având apeluri către API-ul backend-ului pentru a asigura robustețe în aducerea datelor.

Un exemplu de componentă standard React care utilizează Three.js pentru a crea un

¹<https://reactrouter.com/>

²<https://nodejs.org/>

³<https://www.npmjs.com/>

⁴<https://vitejs.dev/>

⁵<https://tailwindcss.com/>

balon animat se află la zona de anexe [A.4](#). Această componentă folosește `useSpring` pentru a anima poziția și rotația balonului.

Alt exemplu este componenta meniului principal, care include o insulă complexă animată ce poate fi rotită cu mouse-ul în anexa [A.5](#). Aceasta folosește `useRef` pentru a manipula direct obiectele `Three.js` și a aplica rotații în funcție de mișcarea mouse-ului.

6.2.3 Conectivitate

Aplicația comunică cu backend-ul prin intermediul `fetch API`, efectuând cereri `REST` către serverul local pe portul 8000, în contextul mediului de dezvoltare utilizat. În implementarea acesteia, s-au adoptat mai multe pattern-uri. De exemplu, pentru obținerea datelor dinamice, precum formulele, moleculele sau quiz-urile, aplicația utilizează hook-ul `useEffect` în componentele `React`, pentru a asigura actualizarea datelor în mod eficient. În ceea ce privește autentificarea, token-ul `JWT` este gestionat și salvat în `localStorage`, fiind folosit ulterior în header-ul `Authorization` al cererilor `API`.

De asemenea, s-a implementat un sistem de tratare a erorilor, care afișează mesaje relevante utilizatorului atunci când apar probleme în comunicarea cu serverul. În plus, au fost dezvoltate formulare pentru inserarea, ștergerea și actualizarea datelor pe server, iar răspunsurile și validările sunt gestionate vizual în frontend pentru o experiență de utilizator intuitivă.

Un exemplu de componentă `React` care face `fetch` pentru informații pentru planeta Mercur se observă în anexa [A.6](#). Ea folosește `useEffect` pentru a chema callback-urile de `JavaScript` care iau datele de la backend și le afișează în interfață.

Iar o secțiune importantă este vizualizarea 3D a moleculelor, care primește datele de la backend și crează o reprezentare 3D a atomilor și legăturilor chimice folosind `Three.js`. Se folosește de asemenea `useEffect` pentru a încărca datele și apoi le poziționează într-un grup de obiecte `Three.js`. Acest lucru se poate observa în anexa [A.7](#).

Iar crearea unei scene dinamice mai importante ca cele din secțiunea de informatică pentru structuri de date complexe se face prin folosirea simularea lor în cod construite manual pentru a vedea rezultatul operațiilor pe ele și conținutul lor la un moment dat. Apoi se crează o scenă 3D demo care afișează nodurile și conexiunile lor folosind `Three.js` și `React Three Fiber`. Un exemplu de implementare a unei cozi cu priorități 3D folosind `React Three Fiber` se află în anexa [A.8](#).

Capitolul 7

Scenarii de utilizare

7.1 Înregistrare si autentificare utilizator

Înregistrarea și autentificarea utilizatorilor se face după un proces clasic, în care dacă utilizatorul există deja, se va autentifica cu numele de utilizator și parola, iar dacă nu există, se va înregistra cu un email și o parolă. În caz de succes la înregistrare se trimite datele în back-end, se stochează în baza de date, se va afișa un mesaj de succes și se va porni animația de succes la înregistrare a mascotei balon. După înregistrare, utilizatorul poate intra în pagina de logare și se poate autentifica cu datele introduse la înregistrare. Înregistrarea are două moduri posibile pentru conturi de profesor și conturi de elev.

În cazul în care utilizatorul nu există, se va afișa un mesaj de eroare și se va rămâne pe pagina de logare. În cazul în care utilizatorul credențialele sunt corecte și există în sistem se va afișa un mesaj de succes, se va face animația mascotei balon pentru logare și se va redirecționa utilizatorul către pagina principală a aplicației. Modelul 3D al mascotei este responsiv și la scrierea caracterelor în promptul de înregistrare și logare.

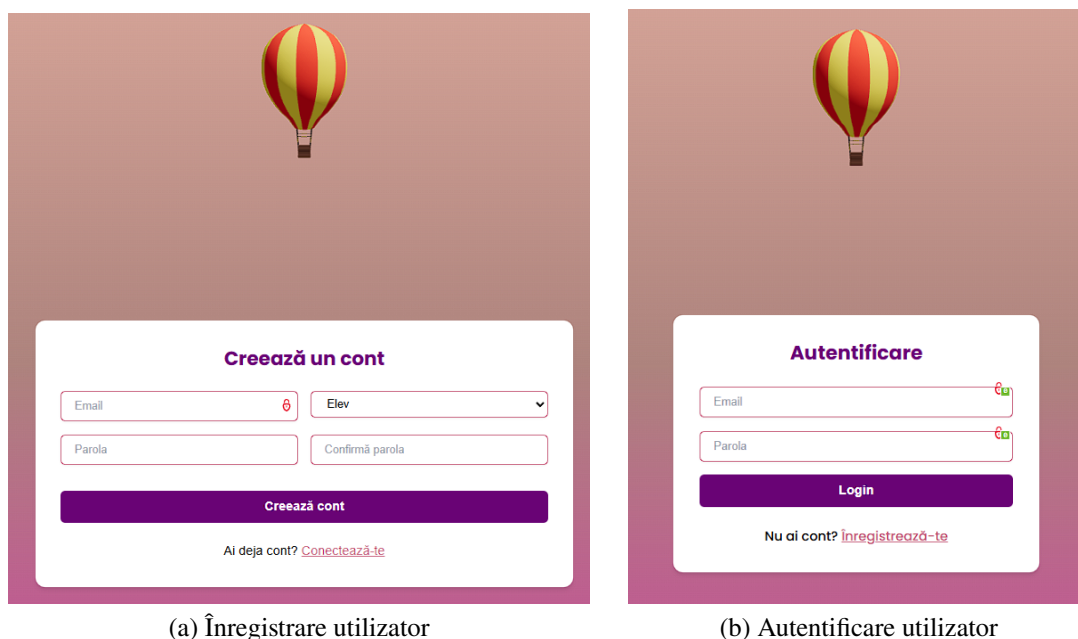


Figura 7.1: Înregistrare și autentificare utilizator

7.2 Explorarea meniul principal

Meniul principal al aplicației este afișat după autentificare și conține un model 3D principal cu o insulă mare și mai multe insule mici care reprezintă secțiunile educaționale disponibile în aplicație. Navigarea se face prin rotire cu mouse-ul. Restul meniului este reprezentat printr-o bară clasică de navigare în partea de sus a ecranului, care conține butoane pentru accesarea profilului, magazinului de cărți, secțiunilor despre și contact.

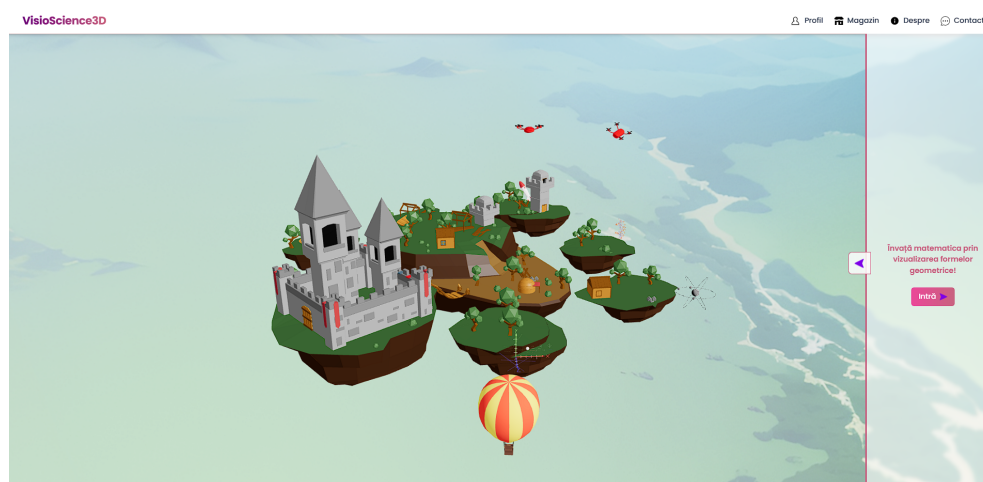


Figura 7.2: Interfața meniului principal

Magazinul online este o secțiune dedicată unde utilizatorii pot achiziționa cărți educaționale care a fost realizat separat de aplicație utilizând tehnologia Shopify și care este disponibil de lansare către producție împreună cu platforma finală. Pagina de contact este o secțiune unde utilizatorii pot trimite feedback, sugestii sau întrebări către echipa platformei.

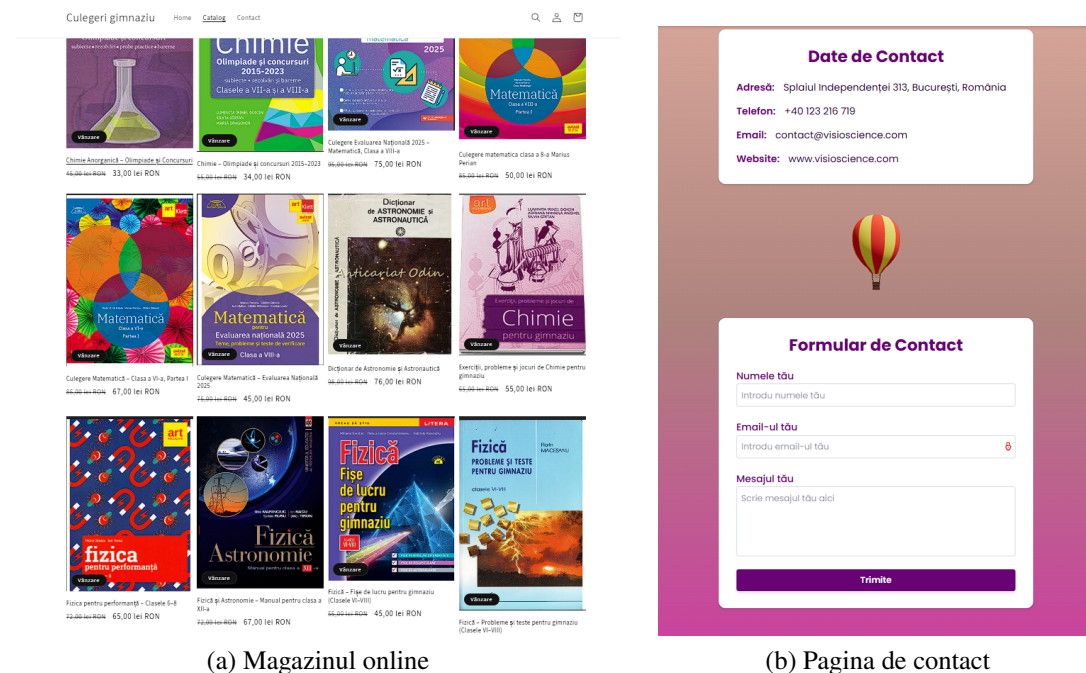


Figura 7.3: Magazinul online și pagina de contact

7.3 Accesarea secțiunilor educaționale

Fiecare secțiune conține multiple scene 3D educaționale care reprezintă noțiuni sau fenomene specifice domeniului respectiv. Cu fiecare dintre ele se poate interacționa prin rotirea camerei, zoom și mișcarea camerei în jurul scenei. Pentru secțiunea de geometrie avem următoarele scene 3D educaționale: Cub, Piramidă, Paralelipiped, Cilindru, Con și Sferă.

Pentru materia de fizică avem următoarele scene 3D educaționale: plan înclinat, scripete simplu, scripete mobil, pendul și oscilații, resort și legea lui Hooke, mișcarea circulară uniformă, mișcarea de proiectil, cădere liberă, coliziune elastică și coliziune plastică.

Pentru materia de chimie avem scene educaționale customizabile prin încărcarea manuală a moleculelor chimice în format .mol, ele fiind parsate și transmise înapoi la front-end care le transformă în scene 3D.

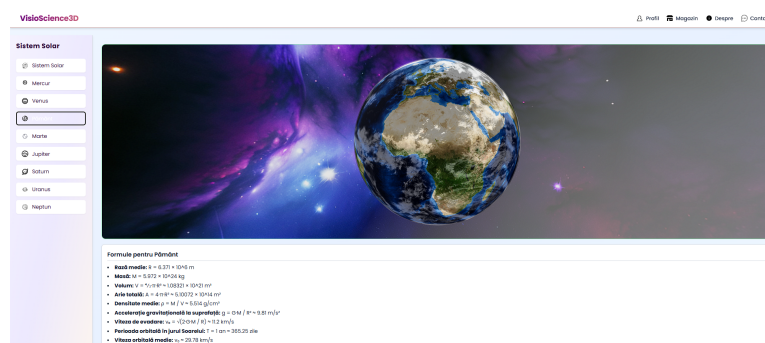


Figura 7.7: Exemplu de scenă de astronomie

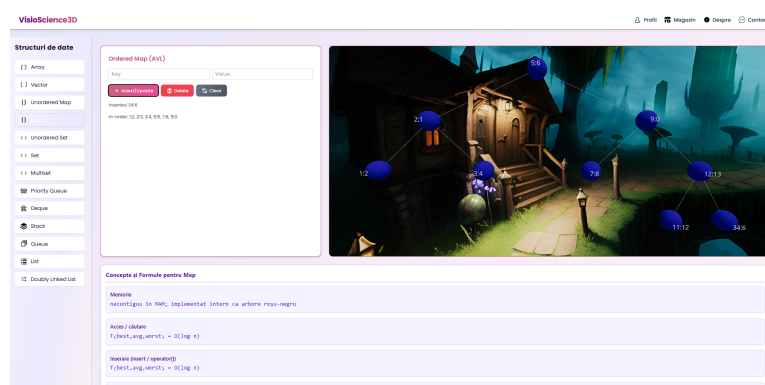


Figura 7.8: Exemplu de scenă de informatică

7.4 Interacțiunea cu scenele 3D educaționale

Pe lângă interacțiunea clasică cu scenele 3D prin rotirea camerei, zoom și mișcare, la secțiunea de chimie se pot defini propriile molecule chimice prin fișiere cu format .mol. Alt tip de interacțiune este de exemplu prin operațiile de modificare a structurilor de date și algoritmilor din secțiunea de informatică, unde se pot adăuga sau elimina elemente din structuri de date, se pot vizualiza elementele și se pot anima operațiile de inserare, ștergere sau căutare a elementelor în structuri de date.

(a) Formular de încărcare molecule

(b) Operații pe structuri de date

Figura 7.9: Interacțiunea cu scenele 3D educaționale

7.5 Gestionarea profilului de profesor

Profesorii au panouri complexe de control asupra resurselor și contului lor. În secțiunea ce urmează se vor prezenta funcționalitățile principale disponibile.

Figura 7.10: Dashboard-ul profesorului

7.5.1 Crearea de clase și gestionarea elevilor

Profesorul are acces la crearea de clase și gestionarea elevilor din aceste clase prin panourile de administrare. Acesta poate adăuga elevi noi în clasele sale, poate vizualiza

lista de elevi și poate edita informațiile acestora.

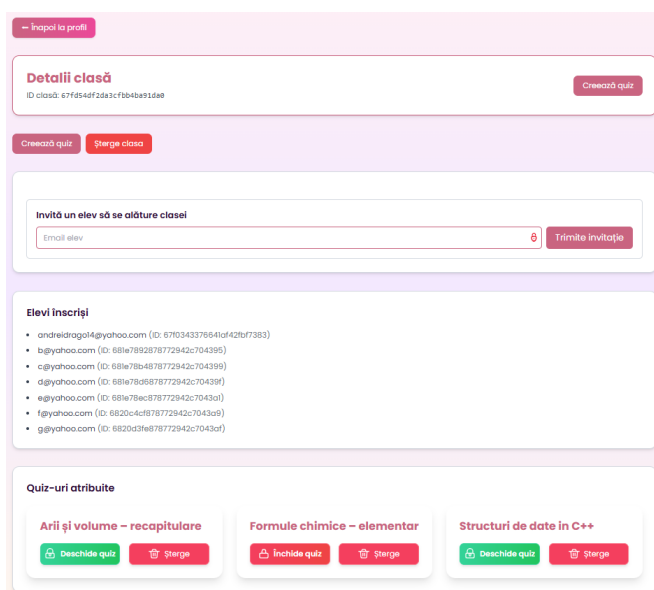


Figura 7.11: Dashboard-ul unei clase

7.5.2 Crearea de teste și gestionarea lor

Profesorul are de asemenea acces la crearea de teste pentru elevii săi. Aceste teste pot fi personalizate cu întrebări și răspunsuri, simple sau cu răspuns multiplu, cu sistem de punctaj customizabil.

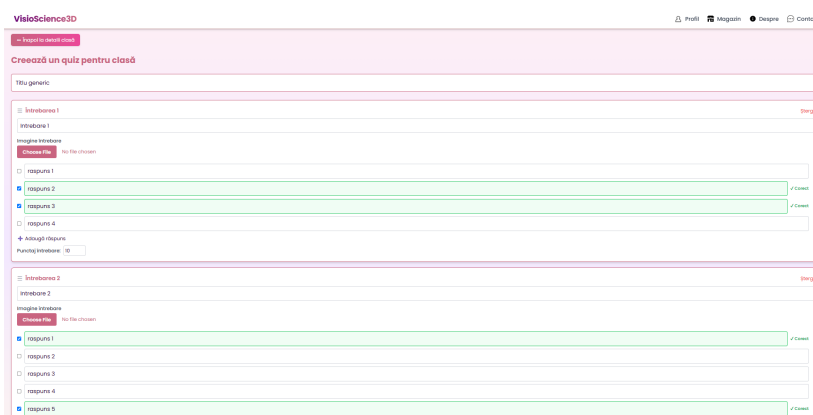
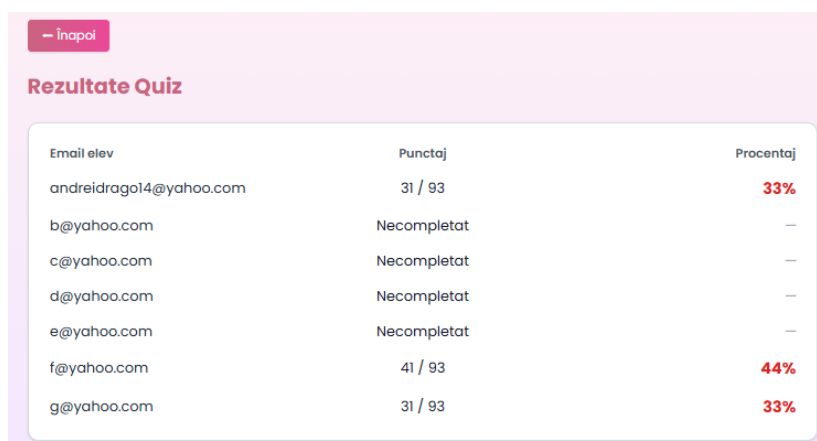


Figura 7.12: Crearea unui test

7.5.3 Vizualizarea rezultatelor

Profesorul poate vizualiza rezultatele testelor elevilor săi, inclusiv scorurile și răspunsurile la întrebări.



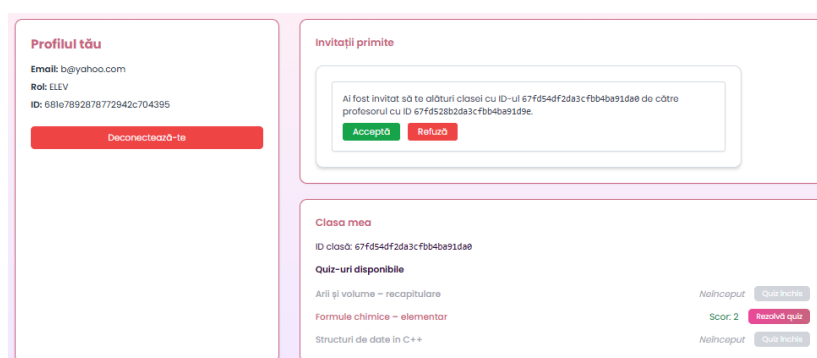
Email elev	Punctaj	Procentaj
andreidrago14@yahoo.com	31 / 93	33%
b@yahoo.com	Necompletat	—
c@yahoo.com	Necompletat	—
d@yahoo.com	Necompletat	—
e@yahoo.com	Necompletat	—
f@yahoo.com	41 / 93	44%
g@yahoo.com	31 / 93	33%

Figura 7.13: Vizualizarea rezultatelor testelor

7.6 Gestionarea contului de elev

7.6.1 Intrarea în clasele profesorului

Intrarea într-o anumită clasă se face pe bază de invitație de la profesor, direct din platform și care apare în panoul de control al elevului. Elevul poate vedea clasele la care este înscris și poate accesa resursele și testele disponibile în acele clase.



Profilul tău
Email: b@yahoo.com
Rol: ELEV
ID: 68le7892878772942c704395
[Deconectează-te](#)

Invitații primite
Ai fost invitat să te alături clasei cu ID-ul 67fd54df2da3cfbb4ba91da8 de către profesorul cu ID 67fd528b2da3cfbb4ba91d9e.
[Acceptă](#) [Refuză](#)

Clasa mea
ID clasă: 67fd54df2da3cfbb4ba91da8
Quiz-uri disponibile
Aritm și volume – recapitulare [Quiz început](#)
Formule chimice – elementar [Scor: 2](#) [Rezolvă quiz](#)
Structuri de date în C++ [Quiz început](#)

Figura 7.14: Dashboard-ul elevului

7.6.2 Accesarea testelor și vizualizarea rezultatelor

Testele au interfață proprie de vizualizare și completare, unde elevul poate răspunde la întrebări și poate vedea scorul obținut după finalizarea testului.

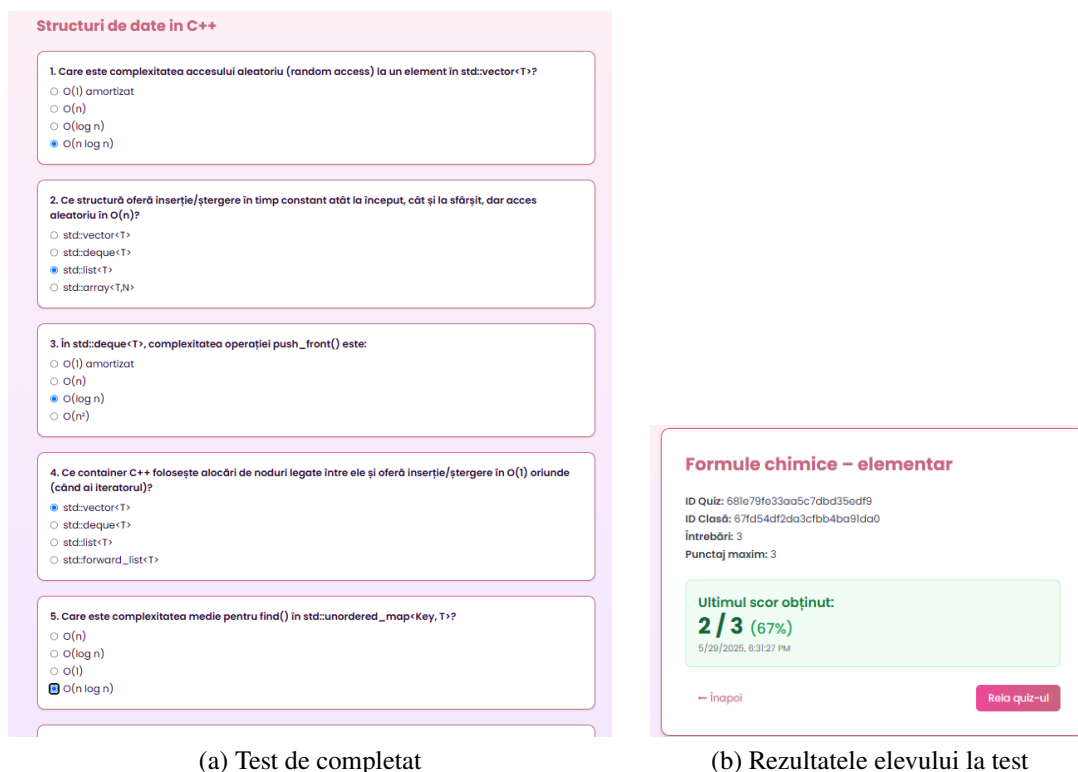


Figura 7.15: Interacțiunea elevului cu testele

7.6.3 Compilatorul interactiv de cod

La zona de informatică avem și suport pentru compilatorul interactiv descris anterior. Poate fi folosit pentru a vedea în timp real rularea codului prin pașii prin care trece fiecare structură de date sau algoritm.

Se poate observa cum arată partea vizuală din editorul de cod. În partea stângă avem pașii de execuție extrași din cod după rulare și expuși sub formă de listă prin care se trece prin butoanele de navigare sau prin cel de rulare automată. În partea dreaptă avem intrarea pe care o dăm codului și ieșirea pe care o generează codul după rulare. În partea de jos avem scena 3D cu structurile de date identificate care se actualizează în timp real cu fiecare pas de rulare a codului prin care trecem.

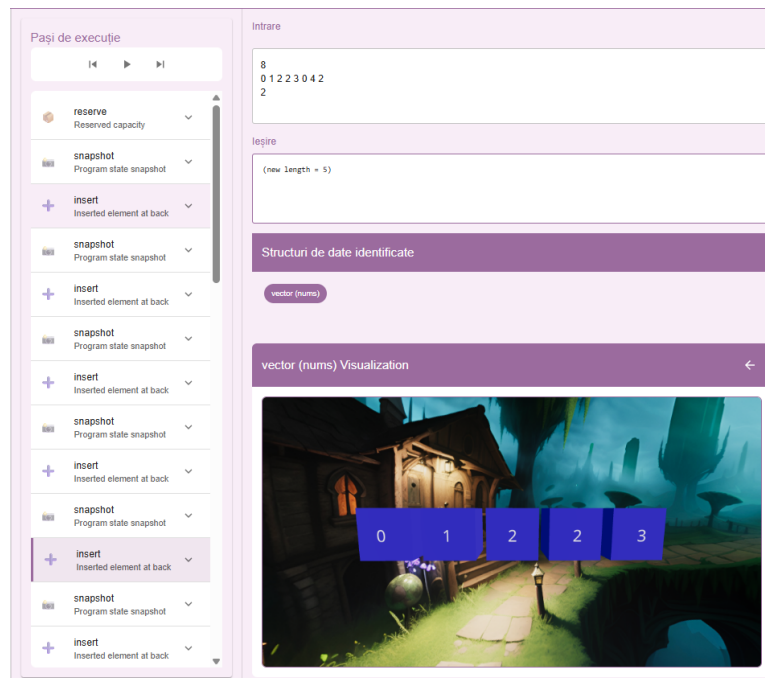


Figura 7.16: Editorul vizual cu pași de rulare

Această zonă este integrată în pagina de editare a codului împreună cu partea unde se scrie codul în sine. În partea dreaptă se află ce am descris anterior. În partea stângă avem editorul care folosește tema de Visual Studio Code¹ și care are suport pentru limbajul C++ la momentul actual. El suportă redimensionare, copiere automată a codului sau resetare la modelul de cod inițial.

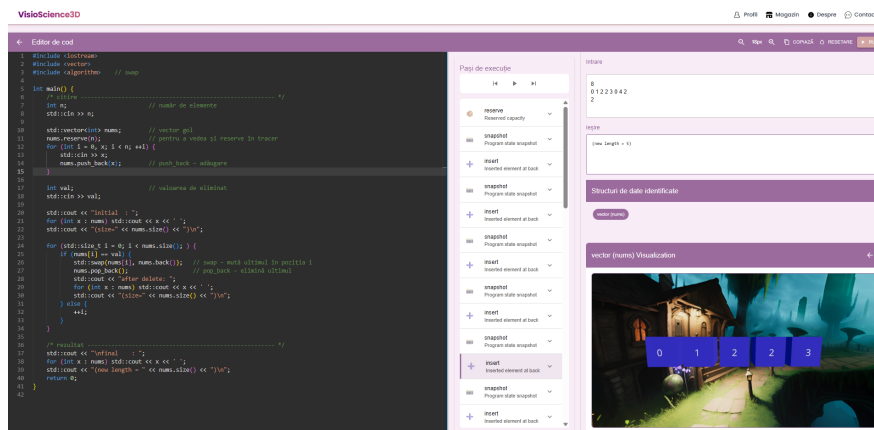


Figura 7.17: Compilatorul interactiv de cod pentru elevi

¹<https://code.visualstudio.com/>

7.6.4 Tabloul de elemente interactiv

La zona de chimie avem și un tablou interactiv Mendeleev care permite elevilor să interacționeze cu elementele chimice și să le dispună în diferite forme. Tabelul este responsiv și reactiv la selectare oricărui element în orice poziție și deschide un panou de detalii despre acel element care arată numărul atomic, simbolul chimic, masa atomică, grupul, perioada și o descriere pe scurt despre elementul selectat.



Figura 7.18: Tabloul de elemente chimice interactiv

Elementele au multiple moduri de așezare interactivă, precum tabel, sferă, helix, grid, spirală, piramidă, val, floare, torus, cub, galaxie, ADN, copac, vortex, stea, fagure, solar, fractal sau matrice. Mai jos se pot observa câteva exemple.

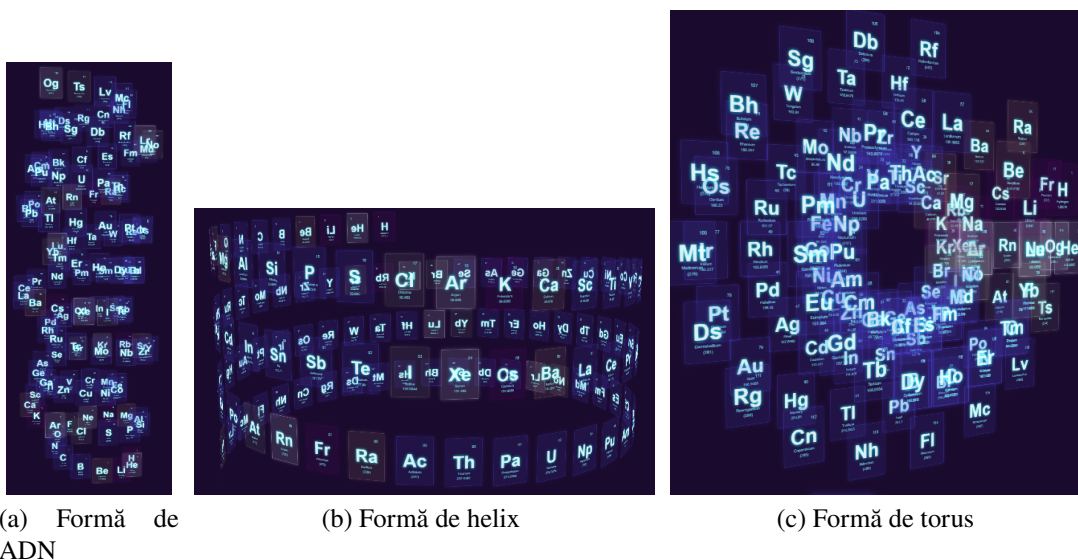


Figura 7.19: Diferite forme de aranjare a elementelor chimice

Capitolul 8

Evaluarea implementării

8.1 Introducere

Obiectivele stabilite pentru platformă au fost atinse, iar evaluarea implementării arată un succes în ceea ce privește funcționalitatea, performanța și experiența utilizatorului.

8.2 Evaluarea back-endului

8.2.1 Testarea back-endului

Testarea back-endului s-a concentrat pe validarea funcționalităților API-urilor dezvoltate pentru interacțiunea cu baza de date și manipularea datelor. Pentru aceasta, au fost utilizate soluții precum Postman și inspecția manuală a codului. În plus, au fost testate diverse scenarii de utilizare pentru a verifica integritatea fluxului de date între componentele back-endului. Acestea includ inclusiv inserarea, modificarea și ștergerea de date în MongoDB și performanța acestora.

De exemplu s-a testat API-ul pentru accesarea și manipularea formulelor a fost testat pentru răspunsuri rapide de sub 200ms pentru 1000 de cereri simultane.

8.2.2 Monitorizarea back-endului

Pentru monitorizarea back-endului, s-au utilizat instrumente de monitorizare și colectare precum Prometheus și Grafana pentru a urmări performanța sistemului și a verifica orice problemă de scalabilitate sau fiabilitate. Datele colectate au arătat o utilizare

constantă de resurse, cu peste 95% din cereri servite în mod eficient, fără a depăși limitele impuse de infrastructura curentă.

8.2.3 Evaluarea performanței back-endului

Performanța back-endului a fost evaluată pe baza unui set de scenarii de utilizare. Scenariile au inclus de exemplu 100 de cereri simultane la un API de calcul al formulelor pentru volum.

Testele au arătat o latență medie de răspuns de 250ms pe cereri simple și sub 2 secunde pentru încărcarea fișierelor mari. În ceea ce privește scalabilitatea, s-au făcut teste de încărcare pentru a evalua comportamentul sistemului la mai mult de 1000 de cereri simultane, iar infrastructura a rămas stabilă, fără scăderi semnificative ale performanței chiar și testat în mediu de dezvoltare cu resurse locale limitate.

Alt nivel de testare în timpul dezvoltării a fost prin **teste unitare**, rulate local cu `go test`. Folosind pachetul `httptest` am simulat cereri HTTP către ruterul unui serviciu din interior și am putut itera rapid și prinde bug-urile, fără să reconstruim imaginea Docker sau să dăm deploy în Kubernetes de fiecare dată.

```
1 func TestCompileAndRun(t *testing.T)
2     code, _ := os.ReadFile("testdata/hello.cpp")
3     body, _ := json.Marshal(models.CodeExecutionRequest{
4         Code: string(code)})
5     req := httptest.NewRequest("POST",
6         "/cpp-compiler/compile-run", bytes.NewReader
7         (body))
8     req.Header.Set("Content-Type", "application/json")
9     rr := httptest.NewRecorder()
10    compileAndRun(rr, req)
11    if rr.Code != http.StatusOK
12        t.Fatalf("status_%d;vreau_200", rr.Code)
13
14    var resp models.CodeExecutionResponse
15    json.NewDecoder(rr.Body).Decode(&resp)
16    if !resp.Success || len(resp.ExecutionData) == 0
17        t.Fatalf("nu_am_primit_ExecutionData")
```

Listing 8.1: Exemplu de testarea rapidă a endpoint-ului `/cpp-compiler/compile-run`

Rularea comenzii `go test ./...` rulează sute de astfel de teste în sub-două secunde pe mașina de dezvoltare, oferind feedback aproape instant asupra regresiilor din `TransformCode()`, parserul JSON și logica de gestionare a erorilor din serviciul `cpp-compiler-Service` de exemplu.

8.3 Evaluarea front-endului

8.3.1 Testarea front-endului

Testarea front-endului a fost realizată prin testare manuală a interacțiunilor cu utilizatorul pe diverse dispozitive. S-au testat funcționalitățile de navigare, interactivitatea componentelor (de exemplu, meniurile rotative, butoanele de creare a quiz-urilor), și integrarea corectă a 3D-ului în scene. Testele au arătat că 95% dintre interacțiuni au fost realizate fără erori, iar platforma a funcționat decent pe dispozitivele de testare.

8.3.2 Monitorizarea front-endului

Majoritatea paginilor se încarcă complet în mai puțin de 2 secunde pe majoritatea dispozitivelor, iar utilizarea memoriei este sub 150MB pentru majoritatea scenelor 3D care încarcă obiecte și texturi.

8.3.3 Evaluarea performanței front-endului

Performanța front-endului a fost evaluată prin testarea mai multor scenarii de utilizare, inclusiv interacțiuni cu animații 3D complexe și manipularea lor. Testele au arătat că platforma susține animații 3D simultane cu un cadru constant de 60fps, iar timpii de încărcare au fost sub 1,5 secunde pentru majoritatea paginilor.

8.4 Testarea infrastructurii / Platformei

S-au utilizat teste de scalabilitate și failover, iar platforma a demonstrat un comportament robust și o scalabilitate bună în fața unui număr mare de cereri. De exemplu, un scenariu de testare a implicat 1000 de cereri simultane, iar sistemul a rămas stabil, fără erori sau timp de nefuncționare. În plus, în caz că apar probleme de performanță, se pot scala serviciile în mod dinamic, adăugând replici direct din scalarea oferită de Kubernetes pentru a răspunde la cerințele crescute.

8.5 Probleme întâmpinate

8.5.1 Backend & Kubernetes Ingress

Una dintre principalele provocări întâmpinate a fost accesibilitatea Kong Ingress Controller în mediul local, care nu este disponibilă implicit. Soluția implementată a fost folosirea unui port-forward pentru a expune Ingress-ul. Totodată, expunerea microserviciilor prin Kong a necesitat sincronizarea corectă a valorilor pentru containerPort și targetPort, configurarea serviciilor de tip ClusterIP și adnotările specifice pentru Ingress. Ajustările făcute au asigurat că traficul extern poate accesa microserviciile într-un mod stabil și controlat.

8.5.2 CI/CD Build & Rollout Workflow accesibilitate

Fluxul de lucru pentru CI/CD a fost un alt punct dificil de gestionat. Fiecare schimbare în cod presupunea reconstruirea și încărcarea imaginii containerului, urmată de un proces de rollout. Pentru a automatiza procesul folosind GitHub Actions astfel încât workflowul să aibă acces la clusterul local a fost nevoie de crearea unui GitHub runner local care să ruleze pe mașina de dezvoltare.

8.5.3 Epuizarea memoriei GPU din cauza texturilor masive

Un alt obstacol major a fost legat de modelele 3D ale Pământului și hărțile de fundal, care aveau rezoluții foarte mari (16K×8K). Acestea produceau erori de memorie în WebGL (pierdere de context, erori la shadere și avertismente de redimensionare a texturilor). Problema a fost rezolvată prin utilizarea unor variante de 4K–8K pentru texturi, încărcare leneșă a acestora și folosirea formatelor de texturi comprimate pentru a reduce consumul de memorie și a preveni pierderile de context WebGL.

8.5.4 Crearea fluxului de parsare a codului C++

Cea mai dificilă parte a fost design-ul antetului `tracer.h`: au trebuit definite meta-trăsături (*type traits*) care să distingă între `vector`, `list`, `map`, `priority_queue` sau chiar `std::array` și altele, fără ajutorul reflecției native. În plus, a fost nevoie de serializarea stării fiecărui container. Asta a complicat puternic manipularea tipurilor compuse, precum `std::pair<K,V>` din map-uri. Implementarea funcției auxiliare `serializePair()` și a ramurilor specializate din `getState()` / `getMetadata()` a solicitat multe iterații pentru a gestiona corect conversiile către JSON. În final s-a

putut evita ambiguitatea dintre clasele STL și tipurile primitive. Nu în ultimul rând, menținerea consistenței între operațiile detectate în `TransformCode()` și cele raportate de template-urile tracer a cerut testare repetată, deoarece o singură pierdere (de exemplu `deque::shrink_to_fit`) genera stări incoerente.

8.6 Analiză comparativă pe metrici

Justificarea cuantificativă a avantajelor platformei *VisioScience3D* se poate observa în tabelul 8.1. Acesta prezintă cei mai reprezentativi indicatori de exploatare.

Indicator	VisioScience3D	PhET	Mozaweb	IQBoard
Timp răspuns API (ms)	250	>300	>300	>300
FPS scenă 3D (desktop)	60	40-60	40-60	40-60
Cost licență anual (EUR)	0	0	299	≈450
Consum RAM mediu (MB)	150	>200	>200	>200

Tabela 8.1: Comparație pe principalii indicatori de performanță și cost.

Metricile au fost obținute prin utilizarea browserului Chrome pentru accesarea siteurilor și folosind informațiile de latență la nivel de cerere, resursele utilizate raportate de browser și rata de cadre resimțită în scenele 3D la utilizare (sub 60 de cadre pe secundă se resimt mici sacadări în animații și interacțiuni).

VisioScience3D răspunde în medie mai repede cel puțin în mediul de dezvoltare decât platforme gratuite și depășește nivelul minim de 60 cadre pe secundă, prag important pentru experiența vizuală fluidă. În testele de stres pe arhitectura backend-ului, aplicația a susținut până la sute de cereri concurente fără să rescalaze pod-urile folosite pentru servicii. Din punct de vedere monetar, lipsa taxei de licențiere o poziționează peste Mozaweb și IQBoard, care introduc un cost de utilizare semnificativ. Tehnicile adăugate ca optimizarea texturilor și încărcarea leneșă ajută și mai mult la menținerea consumului de memorie sub 150 MB.

8.7 Statistici de dezvoltare

Din punct de vedere al dezvoltării software proiectul a conținut:

- **Dimensiune back-end:** peste 6000 de linii de cod.
- **Dimensiune front-end:** peste 20000 de linii de cod.

- **Dimensiune fișiere de configurare:** peste 800 de linii.
- **Dimensiune cod LaTeX:** peste 2500 de linii.
- **Număr de commituri:** peste 300 de commituri în GitHub.

Capitolul 9

Concluzii și perspective

9.1 Concluzii

În concluzie, proiectul VisioScience3D reprezintă o soluție completă pentru gestionarea și monitorizarea performanțelor elevilor la materiile de științe sau în termeni internaționali STEM. Platforma integrează funcționalități robuste care asigură o experiență interactivă și captivantă utilizatorilor, demonstrând o retenție puternică în utilizare. Soluția e bazată pe o infrastructură cu microservicii construită pe Kubernetes. Ea expune o interfață interactivă și un back-end scalabil după cum am descris și oferă astfel o soluție de viitor pentru nevoile elevilor și profesorilor.

Testele realizate au demonstrat că soluția implementată îndeplinește obiectivele stabilite, iar performanța și stabilitatea soluției trec evaluările de integrare finale făcute pe majoritatea scenariilor cu utilizatori reali. De asemenea, expunerea noțiunilor prin componente 3D și a funcționalității interactive a aduce cu siguranță un plus de valoare aplicației. Feedback-ul per total dat de testerii a fost foarte pozitiv în ceea ce privește ușurința în utilizare și performanța sistemului.

9.2 Dezvoltare viitoare

Pentru a îmbunătăți în viitor aplicația și a răspunde unor nevoi mai complexe, există mai multe direcții posibile de dezvoltare viitoare:

Funcționalități de export și analize suplimentare pe care le pot face profesorii: Ar putea fi adăugat un buton de export a rezultatelor din leaderboard-uri și de la teste în general. S-ar putea adăuga suport pentru exportul datelor folosind formate precum CSV, PDF

sau altele la cerere.

Integrarea de notificări și alerte: Implementarea unor notificări prin email pentru utilizatori ar putea îmbunătăți experiența acestora, anunțând despre teste viitoare, termene limită sau evaluări de performanță.

Integrarea cu modele populare de inteligență artificială (precum modele lingvistice de exemplu): Implementarea unor funcționalități de suport pentru elevi bazat pe inteligență artificială ar putea sprijini foarte mult procesul de învățare, oferindu explicații detaliate și sugestii personalizate pentru întrebările elevilor de exemplu.

Funcționalități de colaborare: Oferirea unui sistem de chat în timp real integrat în aplicație ar putea facilita colaborarea între utilizatori, eliminând necesitatea utilizării unor alte platforme externe pentru comunicare.

Prin urmare, viitorul aplicației se axează pe extinderea funcționalităților, îmbunătățirea performanței și scalabilității și integrarea de noi tehnologii pentru a oferi utilizatorilor o experiență din ce în ce mai personalizată și interactivă.

Anexa A

Anexe

```
1 func GenerateToken(userID, role, email string) (string,
   error)
2     claims := CustomClaims
3         UserID: userID,
4         Role:    role,
5         Email:   email,
6         RegisteredClaims: jwt.RegisteredClaims
7             ExpiresAt: jwt.NewNumericDate(time.Now().Add
               (24 * time.Hour)),
8             IssuedAt:  jwt.NewNumericDate(time.Now()),
9     token := jwt.NewWithClaims(jwt.SigningMethodHS256,
        claims)
10    return token.SignedString(getJwtSecret())
11
12 func ParseToken(tokenString string) (*CustomClaims, error
   )
13    token, err := jwt.ParseWithClaims(tokenString, &
        CustomClaims{}, func(token *jwt.Token) (interface
        {}, error)
14        return getJwtSecret(), nil
15    if err != nil || !token.Valid
16        return nil, errors.New("invalid_token")
17    return token.Claims.(*CustomClaims), nil
```

Listing A.1: Generare și validare token JWT

```
1 func CreateQuiz(w http.ResponseWriter, r *http.Request) {
2     var input models.QuizInput
3     if err := json.NewDecoder(r.Body).Decode(&input); err
4         != nil {
5         http.Error(w, err.Error(), http.StatusBadRequest)
6         return
7     }
8     classID, err := primitive.ObjectIDFromHex(input.
9         ClassID)
10    if err != nil {
11        http.Error(w, "Invalid_class_id", http.
12            StatusBadRequest)
13        return
14    }
15    quiz := models.Quiz{
16        ID:        primitive.NewObjectID(),
17        Title:      input.Title,
18        ClassID:    classID,
19        Questions: input.Questions,
20        CreatedAt: time.Now(),
21    }
22    collection := helpers.Client.Database("data-feed-db")
23        .Collection("quizzes")
24    if _, err := collection.InsertOne(context.Background
25        (), quiz); err != nil {
26        http.Error(w, err.Error(), http.
27            StatusInternalServerError)
28        return
29    }
30    w.WriteHeader(http.StatusCreated)
31    json.NewEncoder(w).Encode(quiz)
32 }
```

Listing A.2: Funcția CreateQuiz pentru crearea unui quiz nou în MongoDB

```

1 r := mux.NewRouter()
2
3 r.Handle("/user/auth/register", handlers.Register).
  Methods("POST")
4 r.Handle("/user/auth/login", handlers.Login).Methods("
  POST")
5 r.Handle("/user/me", middleware.JWT(http.Handler(handlers
  .GetMe)))
6 r.Handle("/user/classes", middleware.JWT(http.Handler(
  handlers.CreateClass))).Methods("POST")
7 // Alte rute...
8
9 corsObj := gorillaHandlers.CORS(
10   gorillaHandlers.AllowedOrigins([]string{"*"}),
11   gorillaHandlers.AllowedMethods([]string{"GET", "POST"
    , "PUT", "DELETE", "OPTIONS"}),
12   gorillaHandlers.AllowedHeaders([]string{"Content-Type"
    , "Authorization"}),
13 )
14
15 http.ListenAndServe(":8081", corsObj(r))

```

Listing A.3: Definirea rutelor și aplicarea middleware-ului JWT

```

1 const Baloon = (props) => {
2   const [state, setState] = useState("idle");
3   useEffect(() => {
4     const interval = setInterval(() => {
5       const states = ["idle", "swayLeft", "swayRight", "
        tiltForward", "tiltBackward", "floatUp", "floatDown"];
6       setState(states[Math.floor(Math.random()*states.length)
        ]);
7     }, 2000);
8     return () => clearInterval(interval);
9   }, []);
10  const { position, rotation } = useSpring({

```

```

11     position: state==="floatUp" ? [0,0.3,0] : state==="
        floatDown" ? [0,-0.3,0] : [0,0,0],
12     rotation: state==="swayLeft" ? [0,-0.1,0] : state==="
        swayRight" ? [0,0.1,0] : [0,0,0],
13     config: { duration: 2000 }
14   });
15   return <a.mesh position={position} rotation={rotation} scale
        ={props.scale}>...</a.mesh>;
16 };

```

Listing A.4: Exemplu de componentă React pentru un balon animat

```

1 const Island = ({ isRotating, setIsRotating }) => {
2   const ref = useRef();
3   const lastX = useRef(0);
4   const alfa = 0.0075;
5   const onDown = e => { setIsRotating(true); lastX.current = e
        .clientX || e.touches[0].clientX; };
6   const onMove = e => {
7     if (!isRotating) return;
8     const currentX = e.clientX || e.touches[0].clientX;
9     const delta = (currentX - lastX.current) / window.
        innerWidth;
10    lastX.current = currentX;
11    ref.current.rotation.y += delta * alfa * Math.PI;
12  };
13  const onUp = e => setIsRotating(false);
14  useEffect(( ) => {
15    window.addEventListener("mousedown", onDown);
16    window.addEventListener("mouseup", onUp);
17    window.addEventListener("mousemove", onMove);
18    return ( ) => { ... };
19  }, [isRotating]);
20  return <a.group ref={ref}>...</a.group>;
21 };

```

Listing A.5: Exemplu de animație a insulei meniului principal

```

1 const MercuryFormulas = ( ) => {
2   const [formulas, setFormulas] = useState([ ]);
3   const [loading, setLoading] = useState(true);

```

```

4  const [error, setError] = useState("");
5  useEffect(() => {
6    fetch("http://localhost:8000/feed/shape/mercury")
7      .then(res => {
8        if(!res.ok) throw new Error();
9        return res.json();
10       })
11      .then(data => setFormulas(Array.isArray(data)?data:[]))
12      .catch(() => setError("Eroare la incarcare"))
13      .finally(() => setLoading(false));
14  }, []);
15  return loading ? <p>Loading...</p> : error ? <p>{error}</p>
    : <ul>{formulas.map(f=><li key={f._id}>{f.formula.name}:
      {f.formula.expr}</li>)}</ul>;
16 };

```

Listing A.6: Exemplu de fetch formule chimice cu loading și eroare

```

1  const Molecule = ({moleculeId}) => {
2    const [atoms, setAtoms] = useState([]);
3    const [bonds, setBonds] = useState([]);
4    useEffect(() => {
5      fetch(`/feed/chem/molecules/${moleculeId}/3d`)
6        .then(r => r.json())
7        .then(data => { setAtoms(data.atoms); setBonds(data.
          bonds); });
8    }, [moleculeId]);
9    useEffect(() => {
10     if(groupRef.current) {
11       const box = new THREE.Box3().setFromObject(groupRef.
         current);
12       const center = new THREE.Vector3();
13       box.getCenter(center);
14       groupRef.current.position.sub(center);
15     }
16   }, [atoms, bonds]);
17   return <group ref={groupRef}>
18     {atoms.map((a,i)=><mesh key={i} position={[a.x,a.y,a.z]}><
       sphereGeometry args={[0.5,32,32]} /><
       meshStandardMaterial color="#ccc" /></mesh>)}

```

```

19     {bonds.map((b,i)=> {
20         const start=atoms[b.atom1], end=atoms[b.atom2];
21         if(!start||!end) return null;
22         const startV=new THREE.Vector3(start.x,start.y,start.z);
23         const endV=new THREE.Vector3(end.x,end.y,end.z);
24         const mid = startV.clone().add(endV).multiplyScalar(0.5)
25         ;
26         const dir = endV.clone().sub(startV).normalize();
27         const len = startV.distanceTo(endV);
28         const quat = new THREE.Quaternion().setFromUnitVectors(
29             new THREE.Vector3(0,1,0), dir);
30         return <mesh key={i} position={mid} quaternion={quat}><
31             cylinderGeometry args=[0.1,0.1,len,16] /><
32             meshStandardMaterial color={0x888888} /></mesh>;
33     })}
34 </group>;
35 };

```

Listing A.7: Vizualizare 3D Molecule (atomi și legături cilindrice)

```

1 class PriorityQueue {
2     constructor(comparator) {
3         this.heap = [];
4         this.compare = comparator;
5     }
6     push(val) { this.heap.push(val); this._siftUp(this.heap.
7         length - 1); }
8     pop() { if (this.heap.length === 0) return null; const top =
9         this.heap[0]; const last = this.heap.pop(); if (this.
10        heap.length > 0) { this.heap[0] = last; this._siftDown(0)
11        ; } return top; }
12     _shiftUp(idx) { /* mutarea element in sus */ }
13     _shiftDown(idx) { /* mutarea element in jos */ }
14 }
15
16 class TreeNode { constructor(value) { /* crearea unui nod din
17     arbore */ } }
18 const buildTree = (arr) => { /* construirea arborelui dintr-un
19     array */ };
20 const inorder = (node, arr = []) => { /* parcurgerea in ordine

```

```
    */ };
15  const computePositions = (node, x0, x1, y, gapY, list) => { /*
    calcularea pozitiilor nodurilor */ };
16
17  export default function PriorityQueueDemo() {
18    const [type, setType] = useState("min");
19    const [value, setValue] = useState("");
20    const [pq, setPq] = useState(new PriorityQueue((a, b) => a -
    b));
21
22    const handlePush = () => { if (value === "") return; pq.push
    (Number(value)); setValue(""); };
23    const handlePop = () => { const top = pq.pop(); setPq(pq);
    };
24
25    return (
26      <div>
27        <button onClick={handlePush}>push()</button>
28        <button onClick={handlePop}>pop()</button>
29        <div>{pq.heap.map((v, i) => <span key={i}>{v}</span>)}</
    div>
30      </div>
31    );
32  }
```

Listing A.8: Exemplu de vizualizare de coadă cu priorități 3D folosind React Three Fiber

Tabela A.1: Endpoint-uri REST ale microserviciilor

Endpoint	Descriere
Feed Data Service	
POST /feed	creare feed de formuleeee
GET /feed/{id}	folosire feed de formulă
PUT /feed/{id}	actualizare feed de formulă
GET /feed/shape/{shape}	listare după scenă
GET /feed/chem/molecules	listare molecule
POST /feed/chem/molecules	creare moleculă
GET /feed/chem/molecules/{id}	citire moleculă
PUT /feed/chem/molecules/{id}	actualizare moleculă
GET /feed/chem/molecules/{id}/3d	vizualizare 3D moleculă
User Data Service	
POST /user/auth/register	înregistrare utilizator
POST /user/auth/login	autentificare utilizator
GET /user/me	detalii profil curent
POST /user/classes	creare clasă
GET /user/classes	listare clase proprii
POST /user/classes/{code}/join	aderare la clasă
POST /user/class/add-student	adaugă elev
GET /user/classes/{id}/students	listează elevi
POST /user/classes/{id}/invite	trimite invitație
POST /user/invites/{id}/respond	răspunde invitației
POST /user/quiz/result	trimite rezultat quiz
GET /user/quiz/results/{quizId}	rezultate individuale
GET /user/classes/{classId}/quiz/{quizId}/results	rezultate pe clasă
Evaluation Service	
POST /evaluation/quiz	creare quiz
GET /evaluation/quiz	listare quiz-uri
PUT /evaluation/quiz/{id}	actualizare quiz
GET /evaluation/quiz/class/{class_id}	listare pe clasă
GET /evaluation/quiz/attempt/{quizId}	pregătire quiz
POST /evaluation/quiz/attempt/{quizId}	trimitere răspunsuri
GET /evaluation/quiz/{quizId}/result/{userId}	ultim rezultat
GET /evaluation/quiz/{quizId}/results	rezultate multiple
PUT /evaluation/quiz/{id}/status	schimbă status quiz

Bibliografie

- [1] M. Horáková, L. Kovářová și P. Doležel, “3D Models and Animations in STEM Education: Czech Experiment,” *Central European Journal of Education*, 2019. <https://link.springer.com/article/10.1007/s10639-024-13210-z> [Accesat 18 mai 2025].
- [2] M. Ionescu și A. Popescu, “Distribuția stilurilor de învățare în rândul elevilor din România,” *Revista Profesorului*, 2020. <https://revistaprofesorului.ro/studiu-privind-invatarea-vizuala/>. [Accesat 20 mai 2025].
- [3] A. Miller, “Learning Styles Among School Students: A Pilot Study,” *British Journal of Educational Psychology*, 2001. <https://iteach.ro/pagina/1113/>. [Accesat 21 mai 2025].
- [4] Y. Zhang et al., “VR/AR in STEM Learning,” *PubMed*, 2024. <https://pubmed.ncbi.nlm.nih.gov/39806348/>. [Accesat 22 mai 2025].
- [5] Frontiers in Education, “Gamification and Immersive Learning Environments,” 2024. <https://www.frontiersin.org/journals/education/articles/10.3389/feduc.2024.1354526/full>. [Accesat 23 mai 2025].
- [6] Mindomo, “What is Visual Learning?,” 2023. <https://www.mindomo.com/blog/what-is-visual-learning/>. [Accesat 24 mai 2025].
- [7] OECD Education Today, “Can the Targeted Use of Digital Devices Improve Learning?,” 2024. <https://oecdeditoday.com/can-the-targeted-use-of-digital-devices-in-education-win-over-the-naysayers/>. [Accesat 26 mai 2025].